RetryVLA: Policy-Competence-Guided Recovery Search and Distillation

Progress review · 2026-10-26

SmolVLA on LIBERO-Recover: Spatial (dev) and LIBERO-Object (held-out)

Executive summary

Proven	Competence is what matters: searching for states from which the frozen policy itself can finish (C^pi) beats latent-goal and progress objectives, in search (32 / 2 failures; Object 66 / 8) and after SFT (0.417 vs 0.187, 3 seeds, +0.230 [+0.161, +0.298]).
Best numbers	RetryVLA pi1: test RSR 0.118 → 0.418 ± 0.021 (3 seeds), nominal 0.785 vs 0.796 for pi1-nominal: inside the 2 pt margin, no extra module at test time.
Proven	Distillation is needed: demo-only 0.127, online best-of-8 +0.027 (n.s.), imagination-only (RISE-inspired) 0.068-0.113, residual RL 0.295 but nominal 0.022. Held-out Object: +0.181 over demo-only.
Did not work	The WM is mostly a filter over a proposal library: planning without it solves 0.398 vs 0.458 (M11). On Object, WM data is not better than random-search data at equal count (0.354 vs 0.362).
Next	C^pi v3 from the Cpi-pool labels, more seeds (baselines, Object eps-test), matched-sim-budget distillation (P4), cost accounting (P5), LIBERO-Goal / LIBERO-10. Real-world adaptation: not decided.

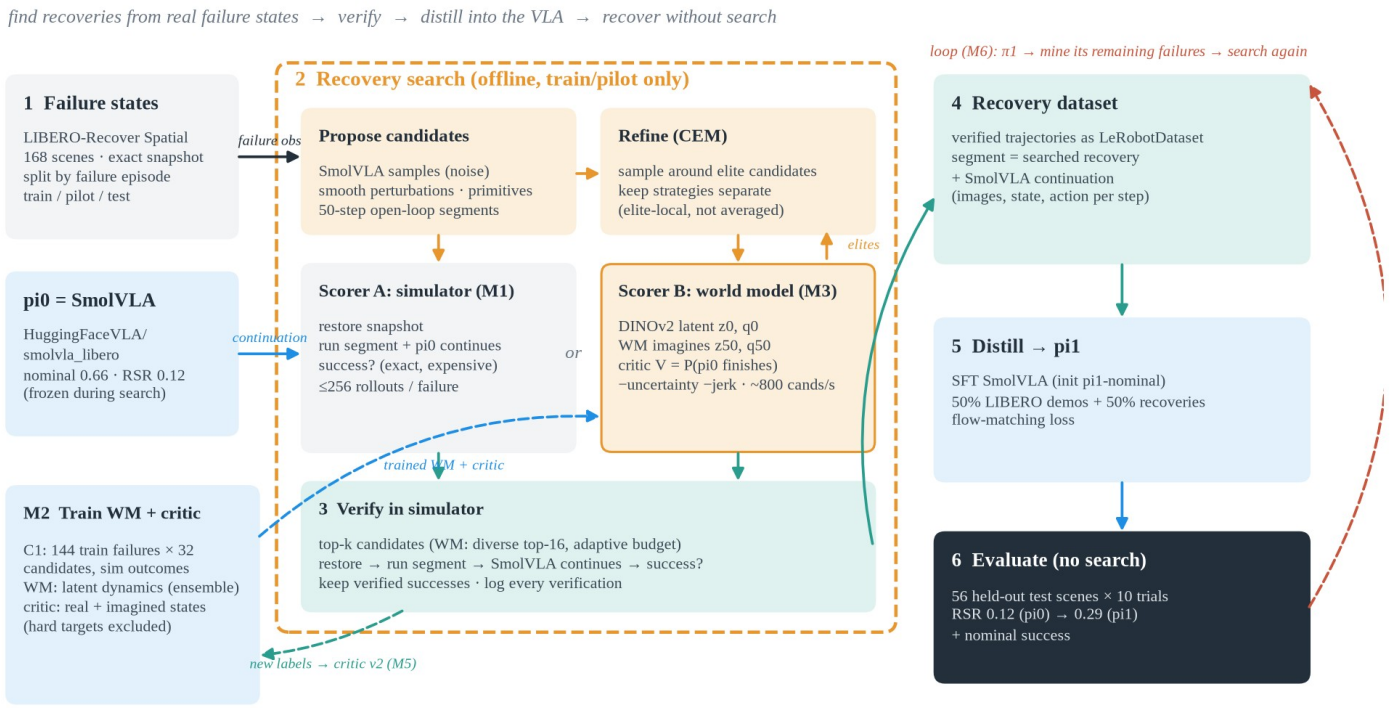
Agenda

1	**Problem and framing**	two modules, pre-registered nominal margin	7	**Nominal preservation**	recovery-ratio Pareto, L_preserve
2	**Timeline M0-M11**	what each milestone answered	8	**Generalization**	LIBERO-Object, frozen recipe
3	**Statistics done right**	cluster bootstrap, 3 seeds	9	**Qualitative**	clickable videos: A / B / C explainers, end-to-end
4	**Q1: why competence?**	search + SFT, critic variants	10	**Related work**	comparison table + method figures
5	**Q2: why a world model?**	cost vs real dynamics, WM as planner	11	**Limitations**	honest negatives
6	**Q3: why distill?**	RISE-inspired, residual RL, best-of-N, demo-only	12	**Plan and decisions**	P-list, scale plan, open questions

Appendix: the full M0-M5 slides (oracle search, world model, critic, M5 search arms and SFT) are kept after the plan.

Problem and framing

Given a pretrained VLA and a compatible world model, can we search for recovery actions that return the robot to states from which the ORIGINAL policy resumes successfully, and distill them into the policy without sacrificing nominal performance?



M-A Policy-aware recovery planner

- frozen pi + WM + failure state → 50-step bridge
- maximize C^pi(ŝ_H) − λ·cost − β·uncertainty
- C^pi trained only on real continuations of pi; verify every bridge in sim

M-B Recovery-preserving distillation

- verified bridges + nominal replay → pi'
- goal: RSR up, NSR(pi') ≥ NSR(pi) − 2 pt
- pre-registered, tested over ≥ 3 seeds

Pipeline as built in M1-M5 (numbers in the figure are the M1-era ones; current pi1: RSR 0.418 ± 0.021)

Timeline: what each milestone answered

milestone	question	answer	key number
M0	can pi0 recover?	almost never	RSR 0.118, nominal 0.656
M1	distill oracle-search recoveries?	yes, ~2× RSR	0.286 (CEM) / 0.279 (random)
M2-M3	latent WM + critic; better than random?	yes, on 53 hard failures	solved 0.36 vs 0.17
M5 / M5b-c	WM data vs random data; search arms	WM diverse wins; round-robin = pi1	solved 0.547 vs 0.363; pi1 0.439 (s0)
M6	loop pi1 → pi2	no gain beyond more training	0.463 vs control 0.480
M7 / M8	weak tasks (stove, cookie box)	no transfer / data skew	M8 0.457 vs 0.504
M9	why competence, WM, distill; 3 seeds	competence ≫ goal; WM ≈ real prefix	pi1 0.418 ± 0.021; 0.417 vs 0.187
M10	LIBERO-Object, frozen recipe	distill + competence replicate	comp 0.354 vs demo-only 0.173
M11	WM plans without the library?	not yet	0.398 vs 0.458 solved
B2-B4 (P2)	policy-grounding or any value?	policy-grounding (pi0 labels)	progress 0.213 vs C^pi 0.480
B3.2 / B3.3	imagination-only; residual RL	below pi0; kills nominal	0.068-0.113; 0.295 / nom 0.022
P3	recovery-nominal Pareto	r 0.5 best inside eps	r 0.7: 0.436 but nominal 0.766

Setting: benchmark, policy, compute

Benchmark

- LIBERO-Recover, Spatial suite
- 168 failure scenes, 84 failed episodes, 9 tasks
- official protocol: 5 closed-gripper + 5 open-gripper trials, ±2 cm object noise, 520 steps
- metric: Recovery Success Rate (RSR); nominal SR on LIBERO-Spatial 10 × 50
- held-out: LIBERO-Object, 26 test scenes, recipe frozen

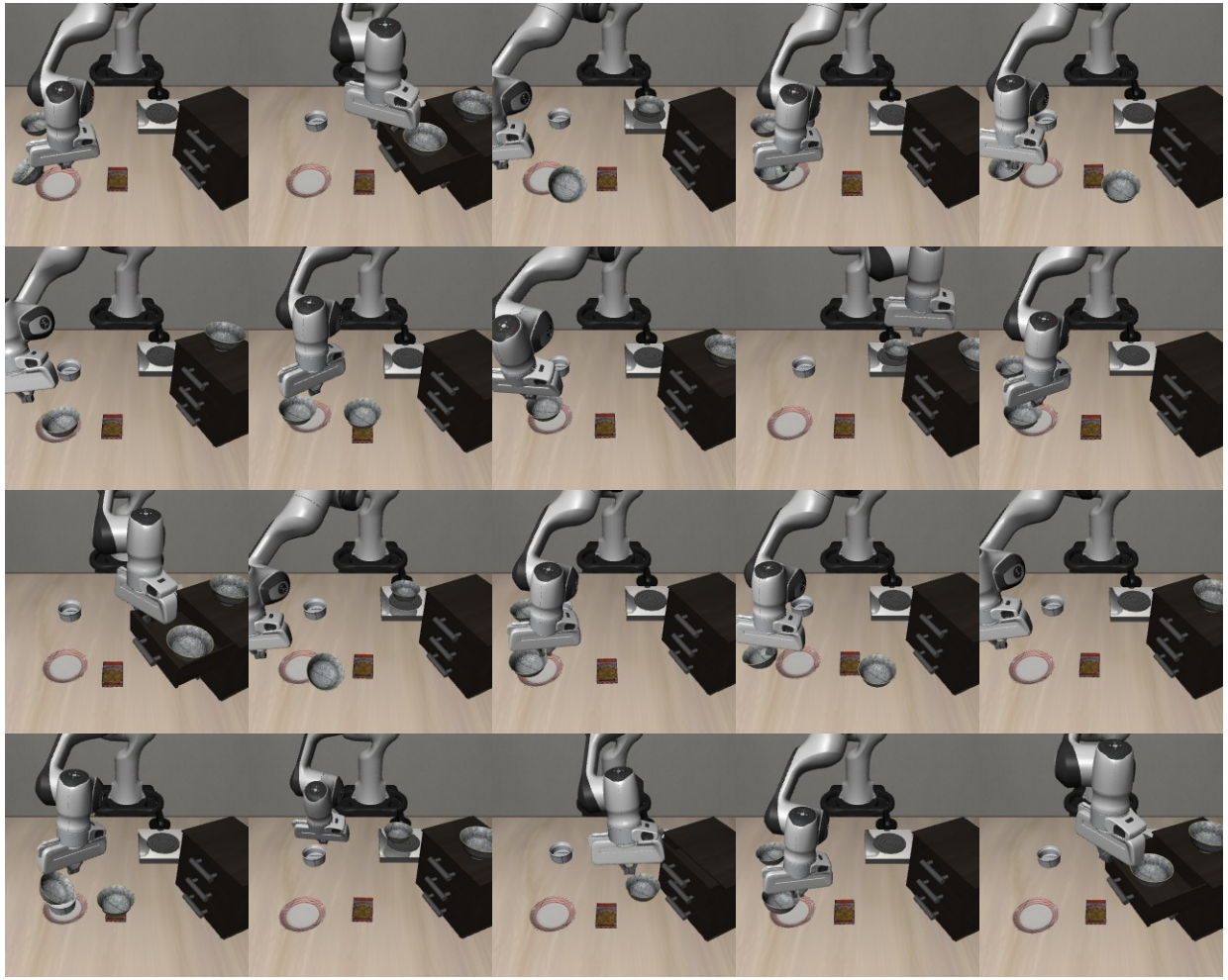
Policy

- SmolVLA, checkpoint HuggingFaceVLA/smolvla_libero (450M)
- chunk 50 actions, replan every 10
- 7-D OSC delta actions, gripper ±1
- pi0 = checkpoint as-is; pi1 = after recovery fine-tuning

Compute

- 1 local A100 + idle A100s borrowed from other shells (avoid-list)
- MuJoCo rendered on GPU (user-local NVIDIA EGL), ~20 ms/step
- bit-exact sim snapshots for branching
- all runs tracked: config, git sha, metrics, registry

What a LIBERO-Recover failure state looks like

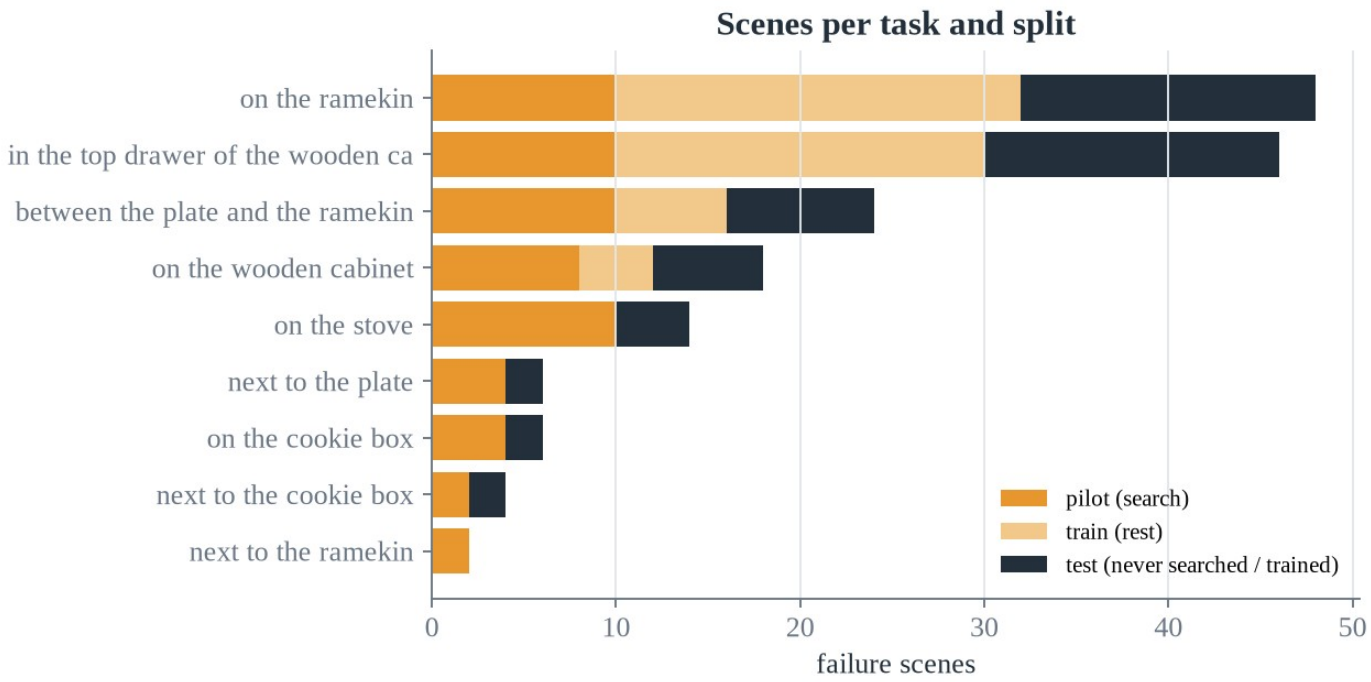


20 restored failure states (agent camera), M0b check: 20/20 deterministic

Each scene = a .bddl task + the exact physics state recorded when SmolVLA was failing

- bowl half-grasped, pushed off the plate, stuck in a drawer, arm in an odd pose
- the gripper is forced closed in half of the trials: an initial state SmolVLA never saw in training
- restore is bit-exact: state, controller, observables. The same actions always give the same pixels
- no L1-L4 difficulty labels released yet

Train / pilot / test split, grouped by failure episode



112

train scenes (search, WM data, SFT)

60

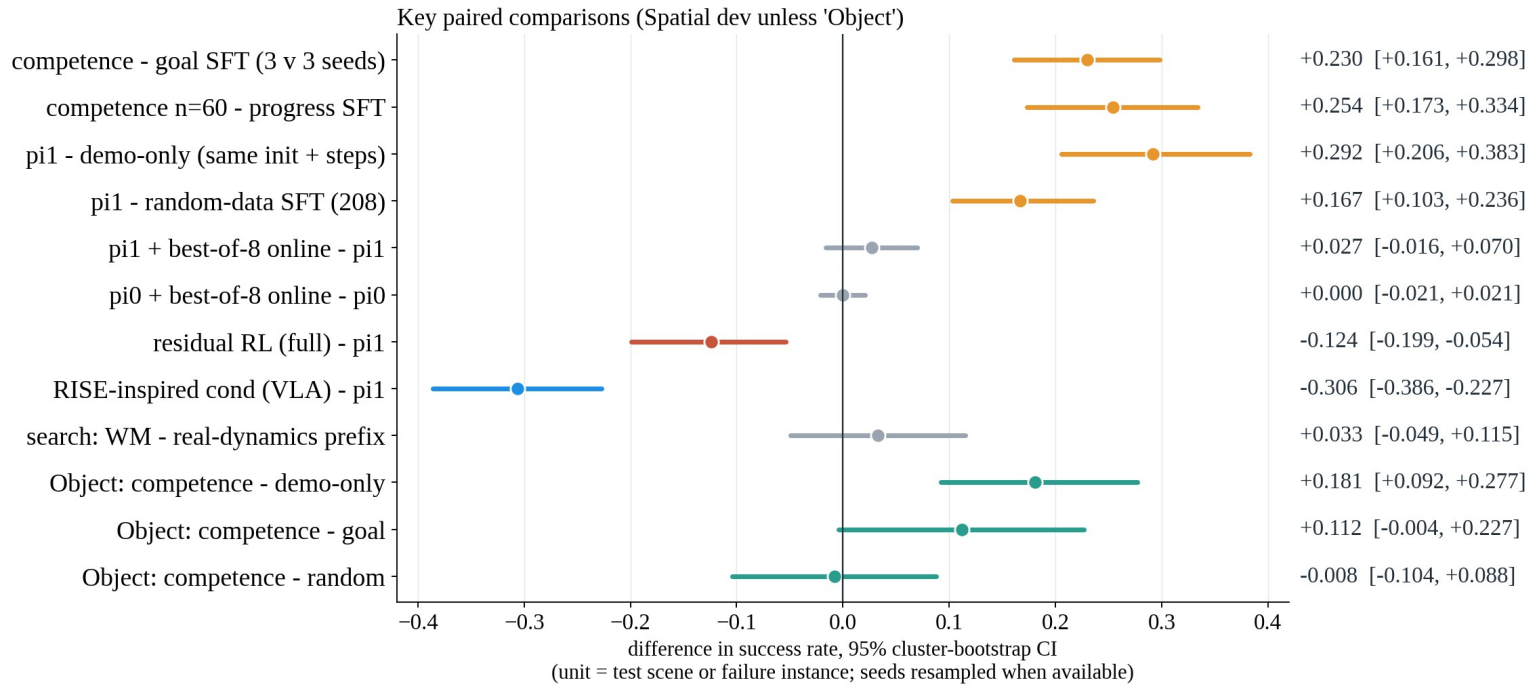
pilot scenes (subset of train, search targets)

56

test scenes: never searched, never trained on

step_0180 and step_0200 of one failed episode are near-duplicates, so they always stay in the same split. Code asserts test scenes never enter search or training.

Statistics: cluster bootstrap over scenes and seeds



Amber = our claims, grey = n.s. checks, red / blue = baselines vs pi1, teal = LIBERO-Object (held-out)

Rule we now apply: differences under ~4 pt RSR on 1 seed are noise; nominal is judged against the pre-registered eps = 2 pt with a 10-task bootstrap.

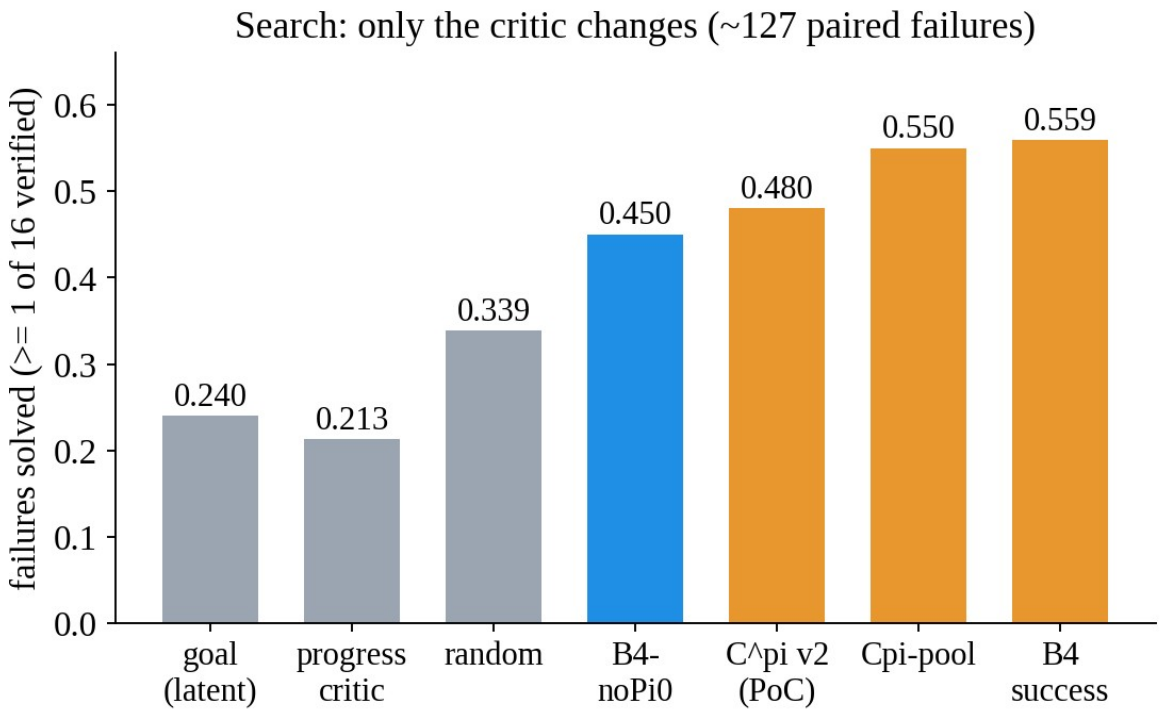
What changed

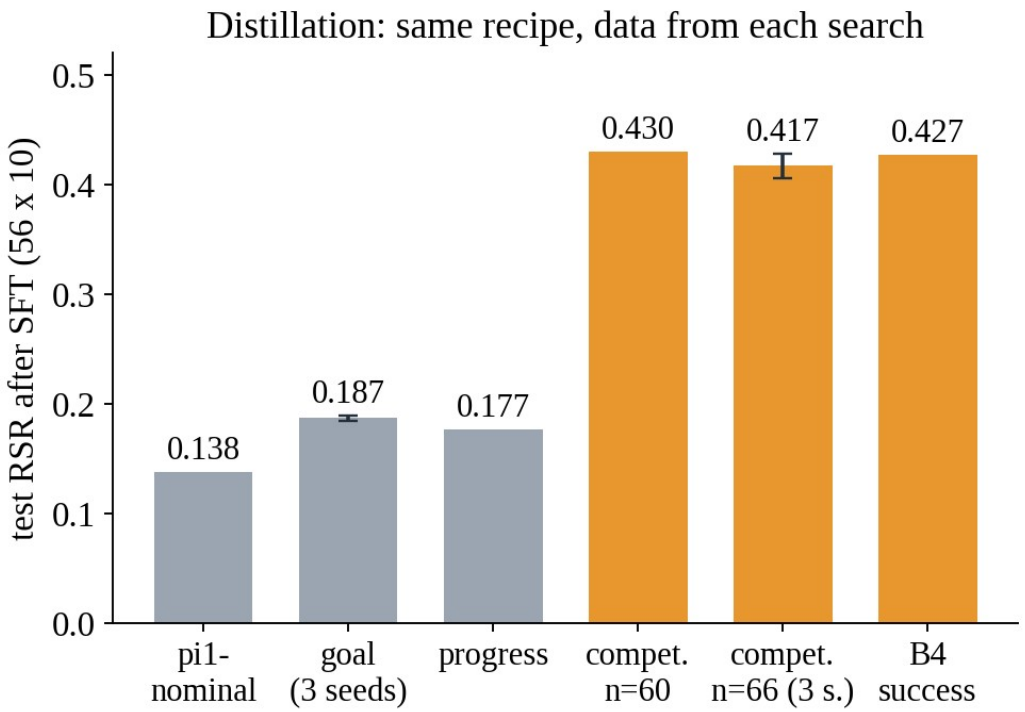
- 10 trials of a scene are not independent: unit = test scene (56) for policies, failure instance for search
- CI = scene × training-seed bootstrap; p = paired sign-flip (policies) or exact McNemar (search)
- effect: M5 WM vs random p 4e-7 (episode level) → 0.003 (scene level); still significant
- 3 seeds: pi1 0.439 / 0.396 / 0.420 = 0.418 ± 0.021; competence and goal SFT, M7 also ×3
- Spatial test is a dev set from M7 on; clean numbers come from frozen Object

Q1: why policy competence?

score a bridge by 'will THIS policy finish from here', not by goal distance or task progress

Q1: competence beats goal and progress, in search and after SFT





amber = trained on pi0 continuation labels; blue = same critic without pi0 labels

Search

C^pi v2 0.480 vs goal latent 0.240, progress critic 0.213, random 0.339 (16 sims / failure). Progress is worse than random.

SFT

competence 0.417 ± 0.011 vs goal 0.187 ± 0.002 (3 seeds), +0.230 [+0.161, +0.298]; 47 / 56 scenes better.

Count-matched

progress data 0.177 vs competence n=60 0.430 (nominal 0.774 vs 0.798); B4-success data 0.427 / 0.796.

Q1: which critic? The gain comes from pi0 continuation labels

critic (ranks WM-imagined candidates)	supervision	n	solved	hits / sim
random (no WM)	-	127	0.339	0.061
goal latent (DINO-WM style)	goal image latent, no critic	125	0.240	0.046
B3 progress	demos + failure 0 + prefix 1	127	0.213	0.038
C^pi v2 (PoC)	pi0 continuations	127	**0.480**	0.156
B4 success	pi0 + pi1 + pi1more + demo (1/4 each)	127	**0.559**	0.182
B4-noPi0	pi1 + pi1more + demo, no pi0 labels	129	0.450	0.093
Cpi-pool	B4's pi0 labels only (7,160), B4 trainer	129	**0.550**	0.156

Same pi0 failures, proposal pool, WM v1 and 16 verified sims; only the critic changes

Question answered: does B4 win because it is policy-agnostic, or because of more / re-filtered pi0 continuation labels? The latter.

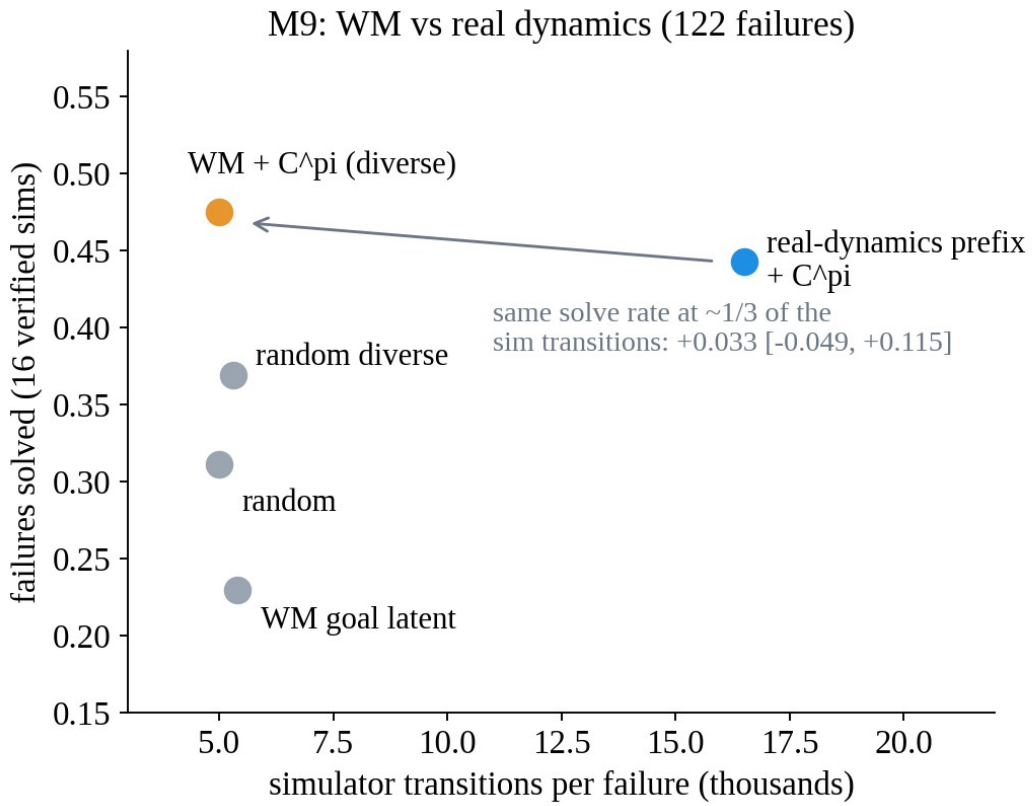
Reading

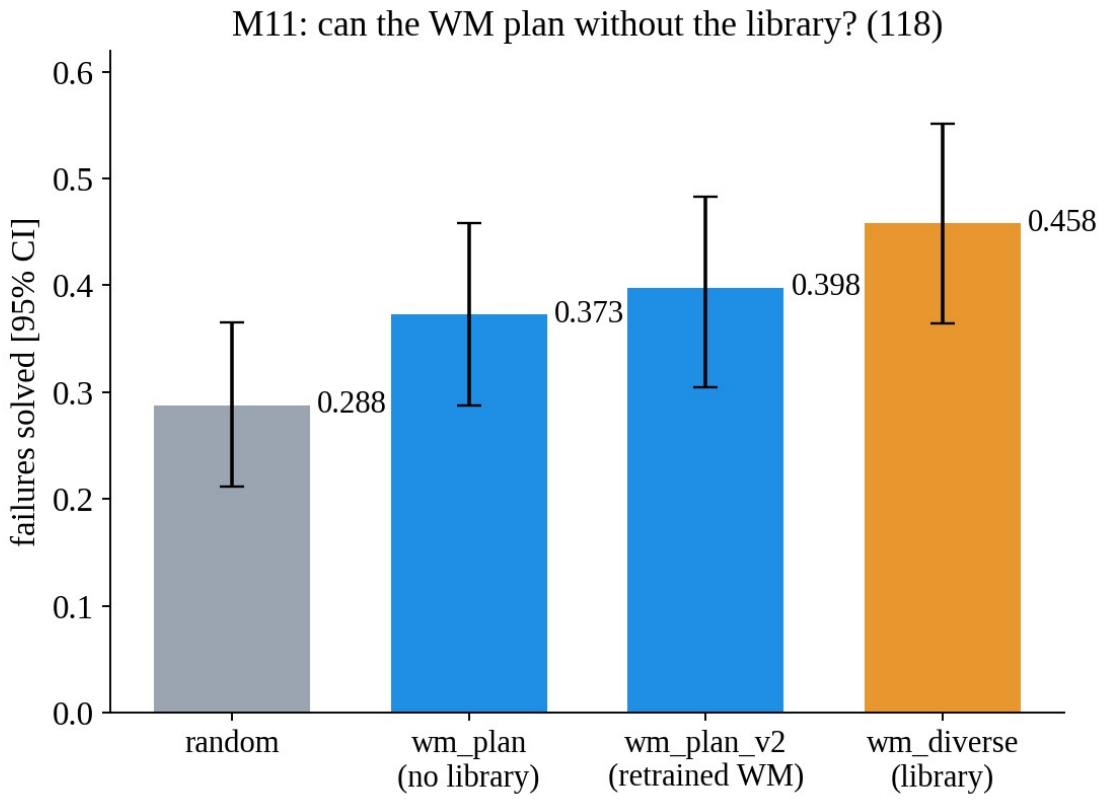
- B4 − B4-noPi0: +0.119 [+0.048, +0.198], p = 0.004: drop pi0 labels and the gain is gone
- Cpi-pool ≈ B4 (0.540 vs 0.556, p = 0.82): pi0 labels alone recover it
- labels from other policies and demos add nothing; progress supervision < random
- so B4 is a better competence critic, not evidence for policy-agnostic value → candidate C^pi v3

Q2: why a world model?

honest answer: acquisition efficiency (more verified recoveries per simulator rollout), not a planner yet

Q2: the WM buys efficiency; it does not plan well on its own yet





What the WM buys

Same solve rate as a real-dynamics prefix at ~1/3 of the transitions (5.0k vs 16.5k / failure); 3.1× (Spatial) and 2.2× (Object) verified hits per sim vs random.

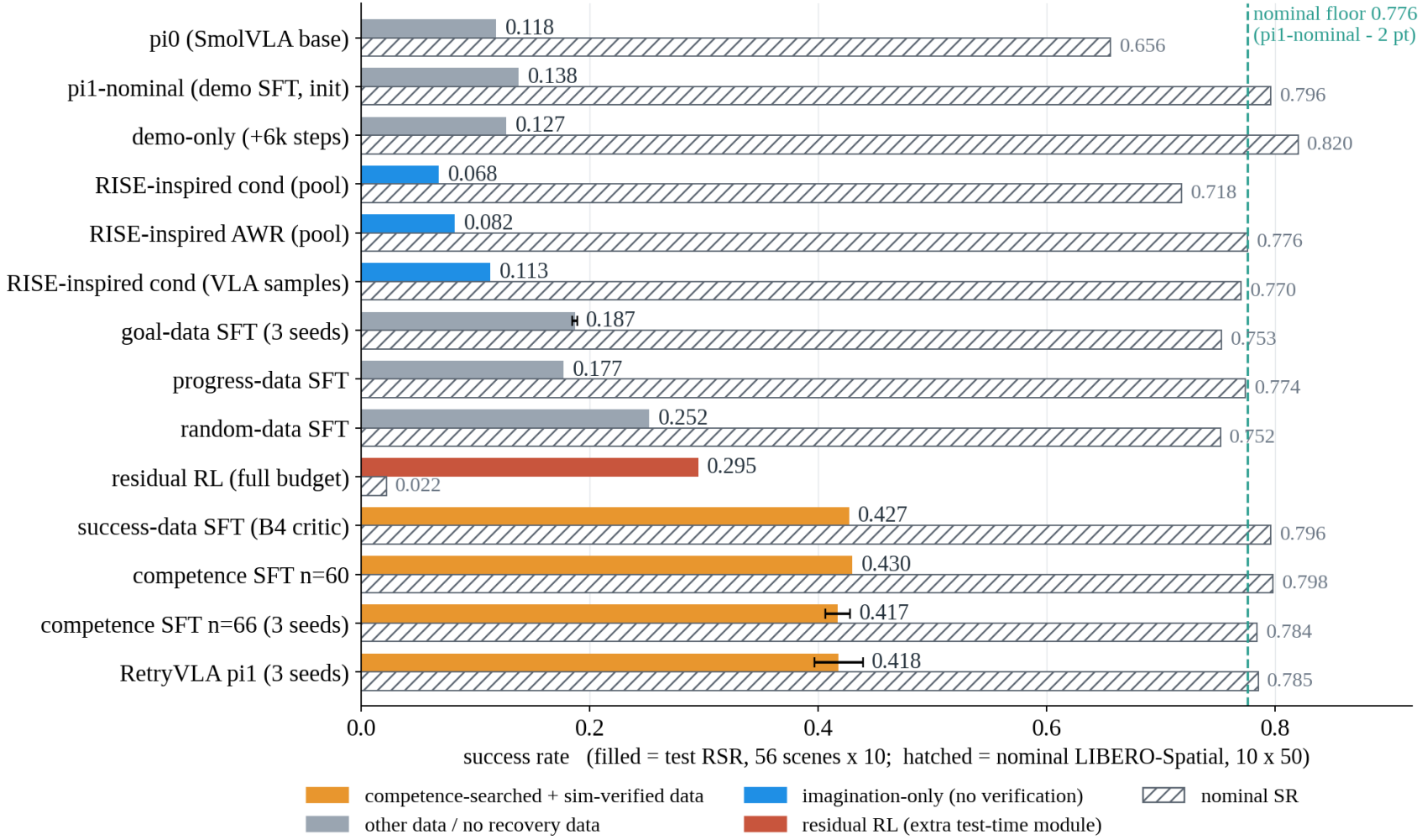
What it does not (M11)

No primitives: −0.175 vs library (p = 3e-4). Planning from noise: 0.398 vs 0.458 (n.s., −0.64 hits / failure, p = 0.02); only 2 failures solved outside the library. WM = mostly a filter.

Q3: why distill?

every alternative at the same budget is below pi1, or pays with nominal success or a test-time module

Q3: all baselines vs RetryVLA (Spatial dev split)



0.418

RetryVLA pi1 test RSR (3 seeds, ± 0.021), nominal 0.785

- only competence-searched, sim-verified data reaches pi1's RSR AND stays above the nominal floor
- goal, progress and random data all fall below the floor
- no extra module at test time

Q3: four ways to avoid distillation, and why they fail

Demo-only SFT

0.127

test RSR
nominal 0.820

same init + steps, no recovery data: −0.292 [−0.380, −0.207] vs pi1. Recovery data is what helps.

Online best-of-8

+0.027

test RSR
on pi1 (n.s.); pi0 +0.000

WM + critic pick among 8 VLA samples every replan: costly at test time and only re-ranks the policy's own samples: no gain at all on pi0.

Imagination-only (RISE-inspired)

0.068-0.113

test RSR
nominal 0.718-0.776

learn from imagined advantage, no verification: imagined positives V 0.81 vs real success 0.185 (~4 / 5 wrong).

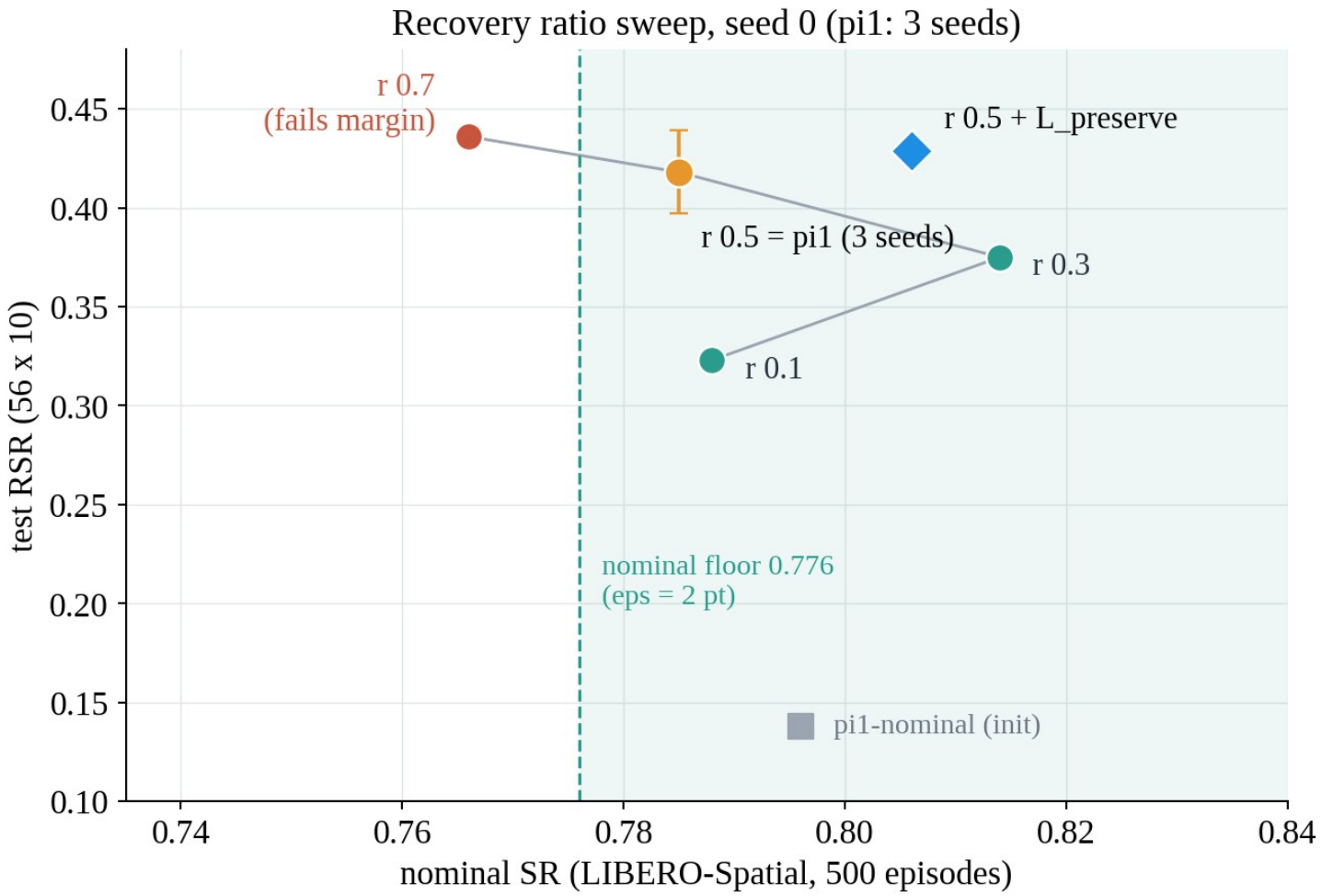
Residual RL (PPO bridge)

0.295

test RSR
nominal 0.022

full matched budget 3.47M: learns recovery but fires on every episode start (25% budget: 0.111 / 0.400).

Recovery-nominal Pareto: r 0.5 is the best point inside the margin

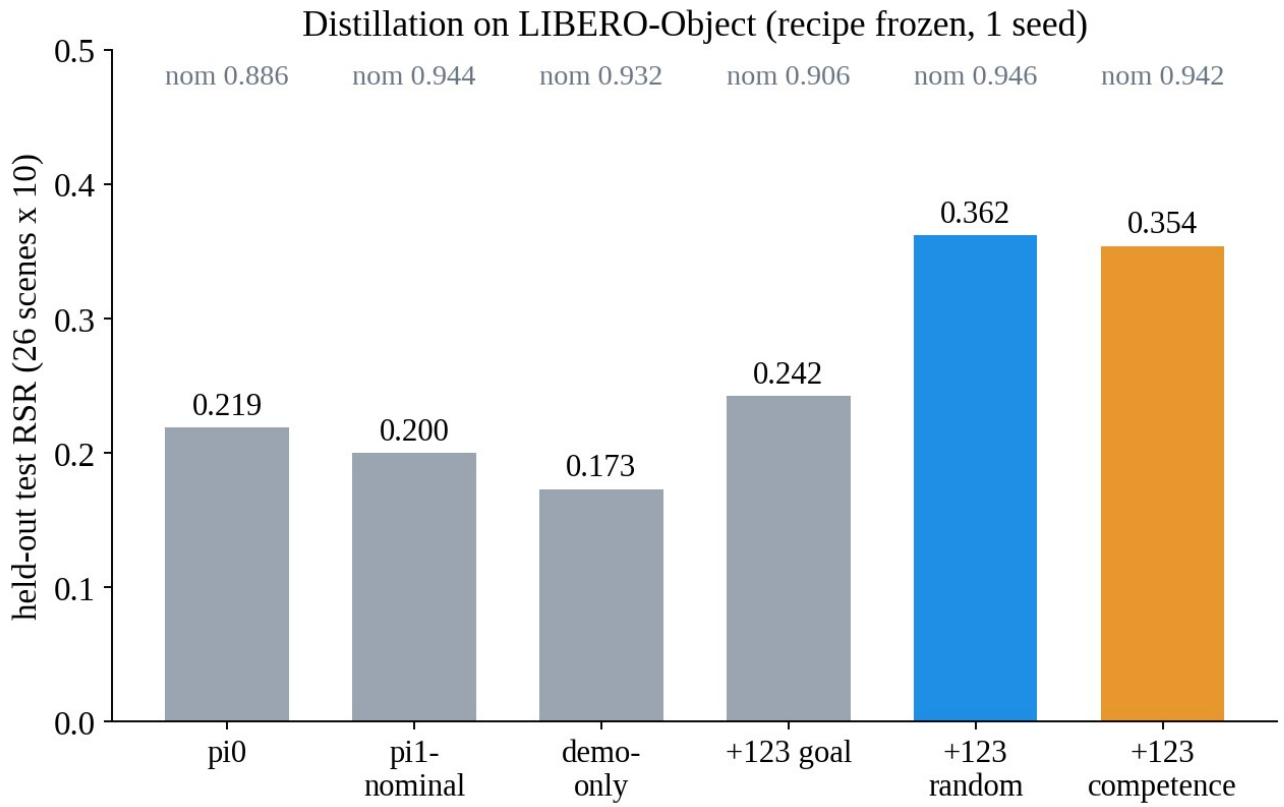


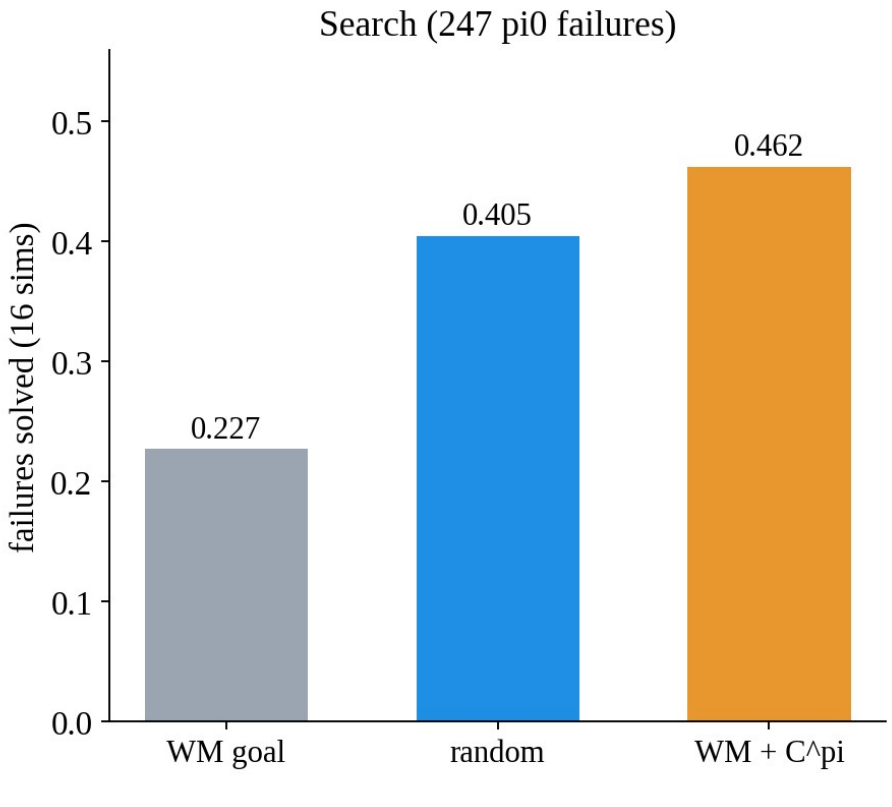
P3 sweep, seed 0 per point; pi1 = r 0.5 (3 seeds). Includes the final r 0.3 / r 0.7 nominal evals

Reading

- RSR rises with the recovery ratio: 0.323 / 0.375 / 0.418 / 0.436 for r 0.1 / 0.3 / 0.5 / 0.7
- r 0.7: +0.017 RSR (n.s.) but nominal 0.766: fails the 2 pt margin
- r 0.1 / 0.3 keep nominal (0.788 / 0.814) but lose RSR (−0.095, −0.043)
- L_preserve: 0.429 / 0.806, inside pi1's seed spread (0.396-0.439 / 0.766-0.808): not needed
- pi1 dNSR −0.011 [−0.046, +0.024]: constraint met

LIBERO-Object (held-out): what replicates and what does not





Replicates

competence − demo-only +0.181 [+0.092, +0.277], p = 0.001; nominal 0.942 vs 0.944 (1 seed).

Replicates in search

competence vs goal 66 / 8 failures, p = 2e-12; SFT +0.112 [−0.004, +0.227], p = 0.08 (directional).

Does not replicate

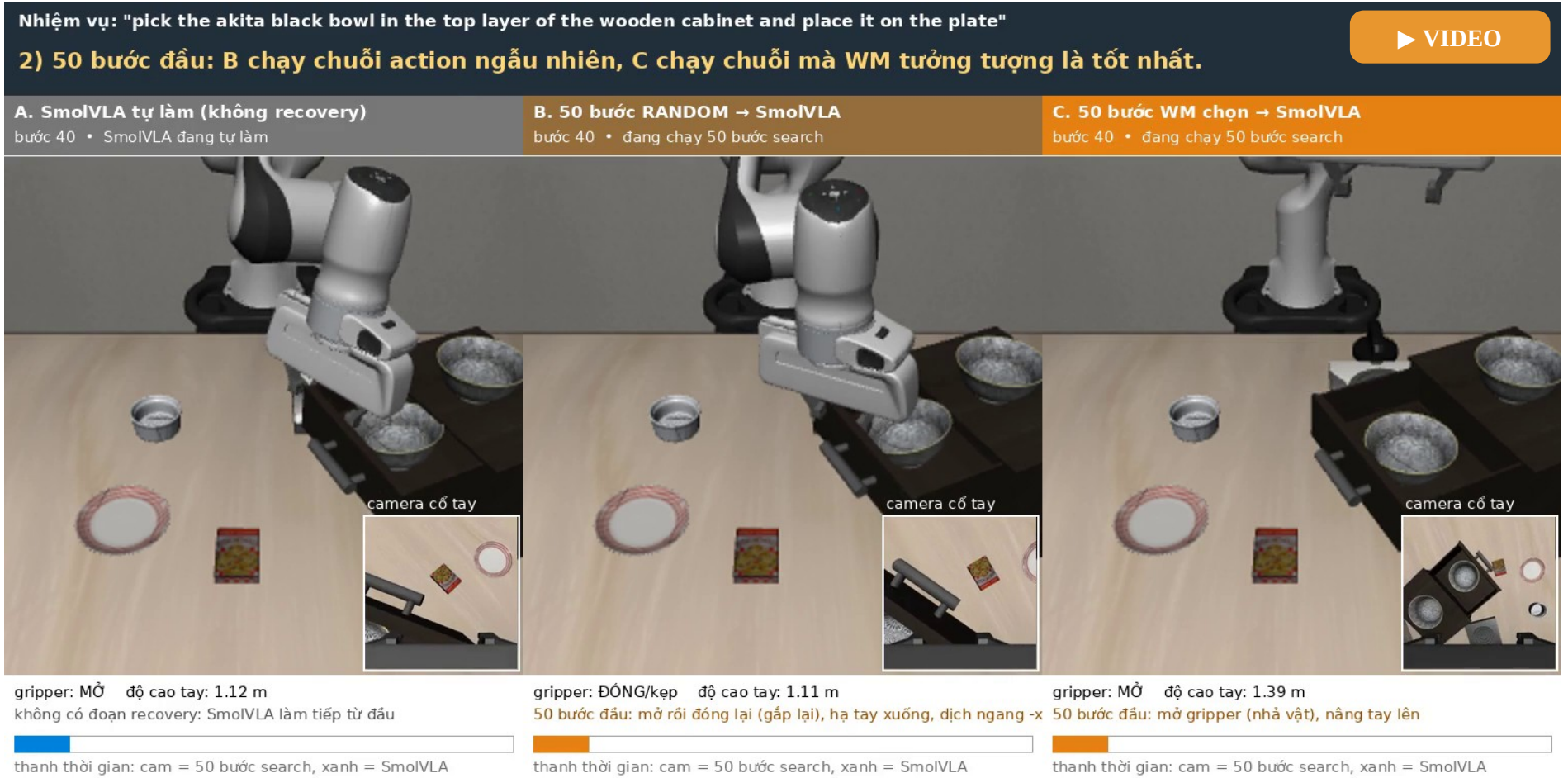
competence vs random data at equal count −0.008 (p = 0.93). Claim → acquisition efficiency: 0.134 vs 0.060 hits / sim.

Qualitative: what the recoveries look like

A = SmolVLA alone, B = random search, C = WM + C^pi search; then the distilled pi1 with no search

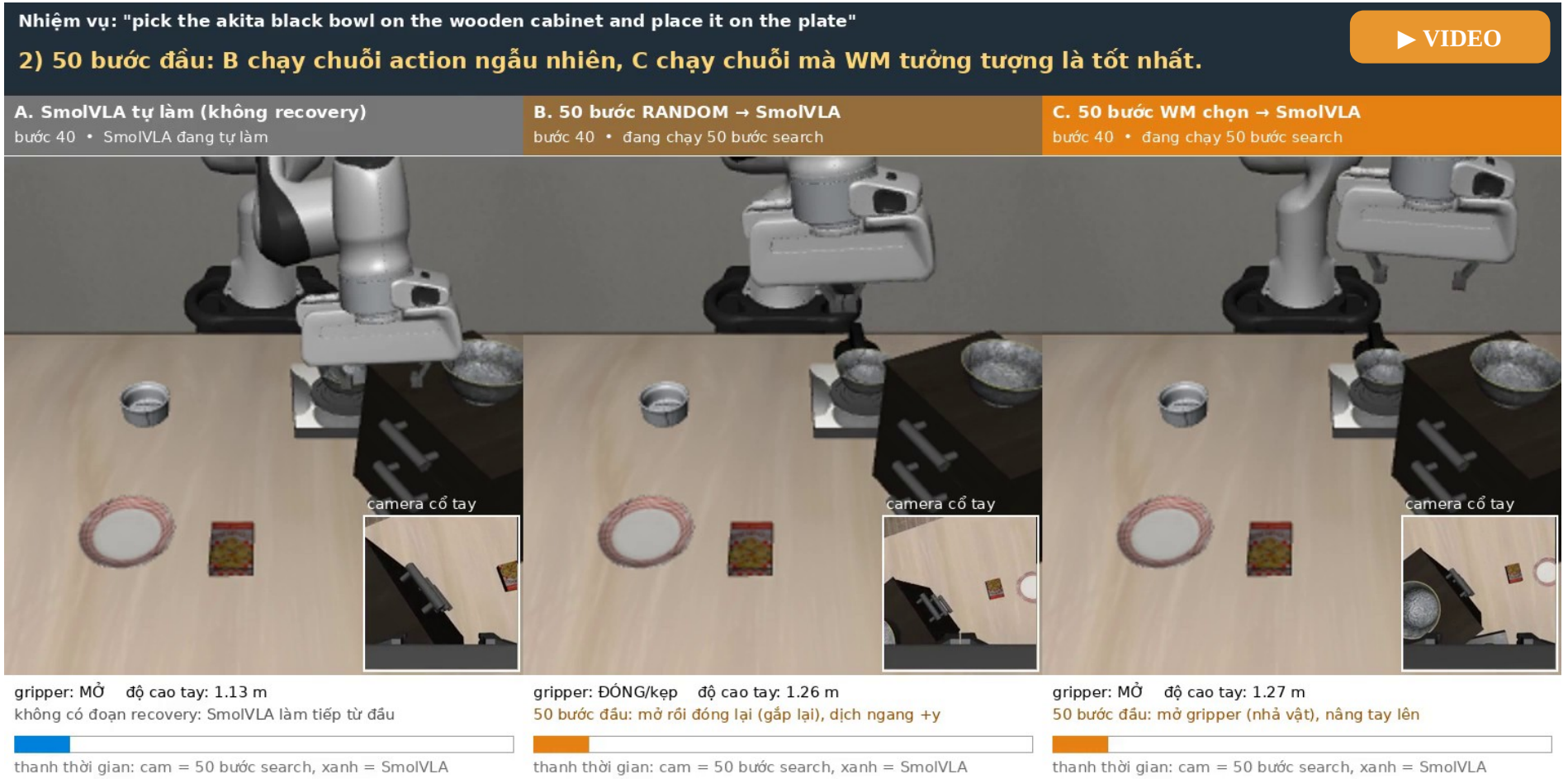
videos are clickable: click to play in PowerPoint

Video: drawer failure, A alone / B random / C WM



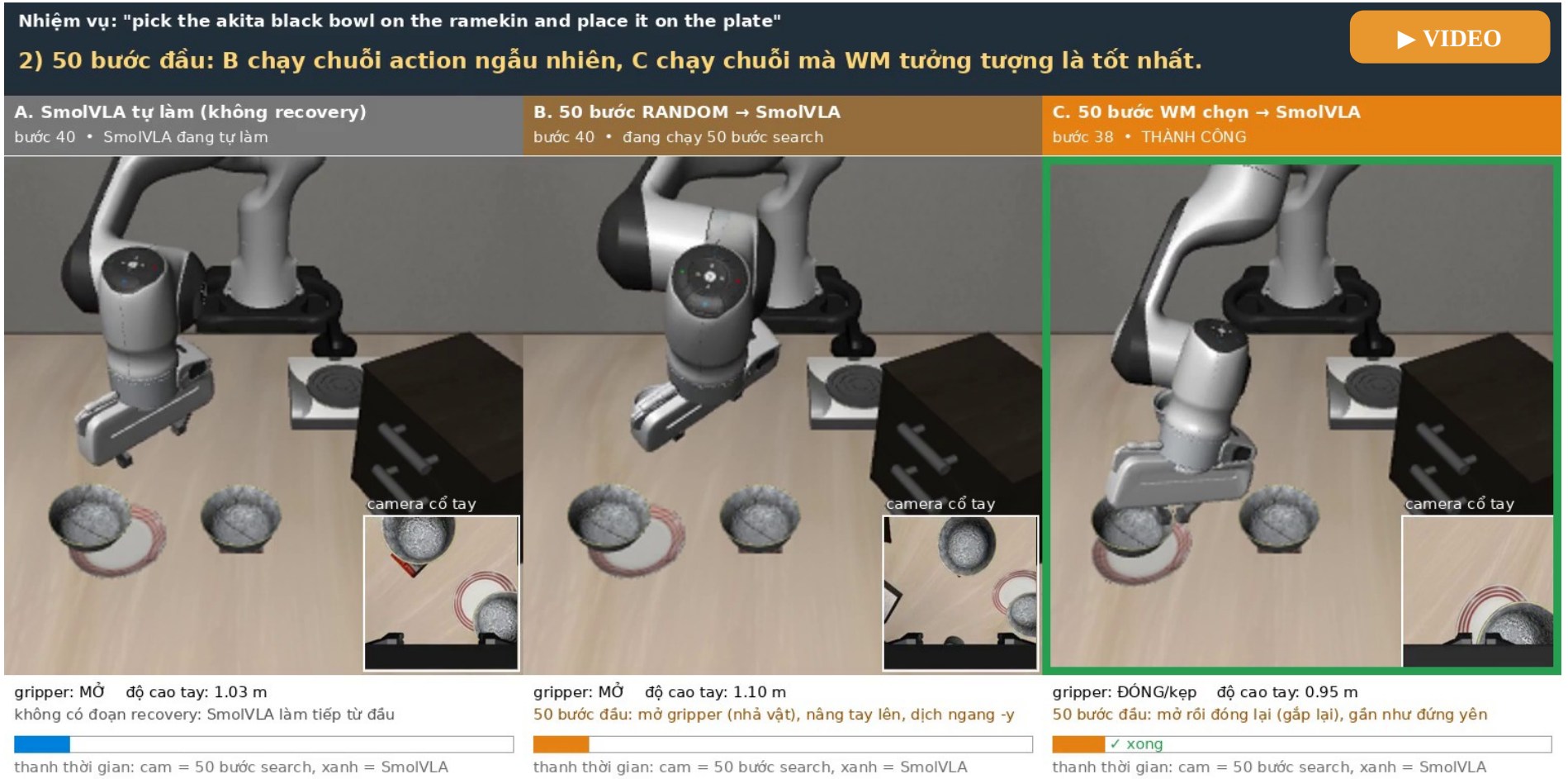
All 3 columns start from the SAME failure state. Only the first 50 steps differ; after that SmolVLA drives every column. C lifts 1.13 → 1.38 m with the gripper open (orange bar), then SmolVLA re-approaches, grasps and places the bowl at step 198. A and B hover at the drawer for 360 steps.

Video: cabinet failure, A alone / B random / C WM



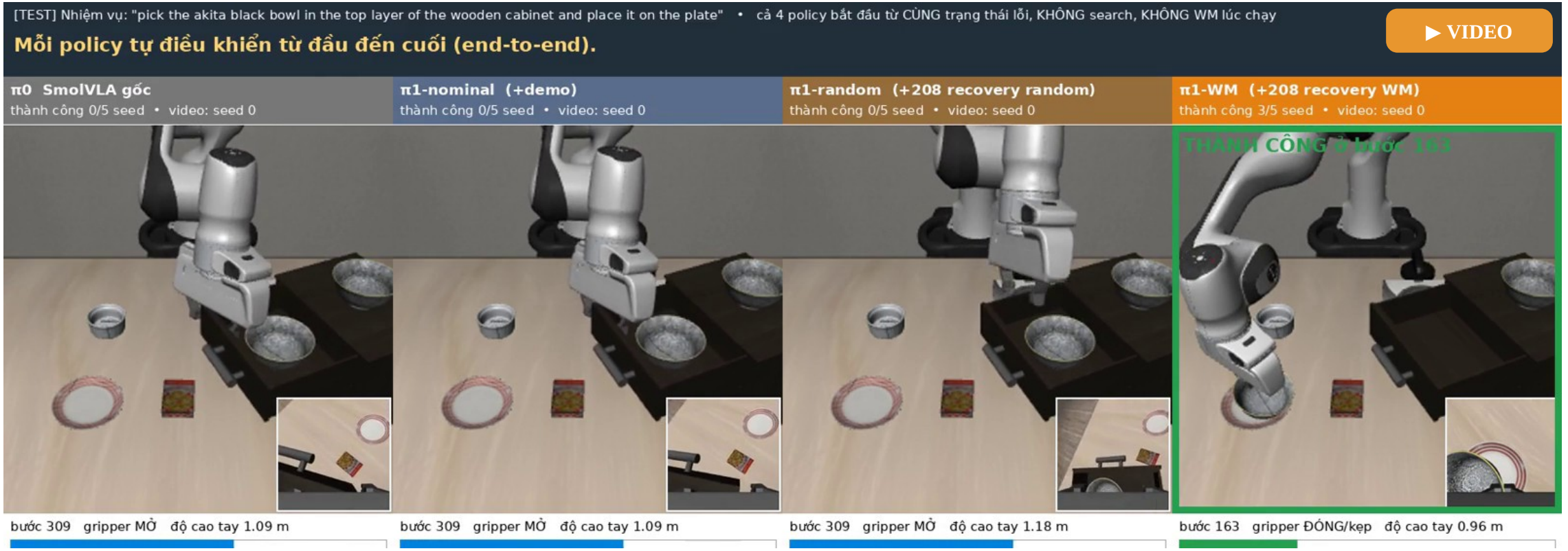
All 3 columns start from the SAME failure state. Only the first 50 steps differ; after that SmolVLA drives every column. B tries to re-grasp in place (open → close, shift +y) and fails. C lets go and lifts; SmolVLA then finishes at step 142.

Video: ramekin failure, A alone / B random / C WM



All 3 columns start from the SAME failure state. Only the first 50 steps differ; after that SmolVLA drives every column. Here the WM segment alone finishes the task (step 39): a short grasp-in-place the WM predicted to work. Random 0/16 on this failure.

Video: end-to-end, no search, unseen TEST failure



0/5 π0 SmolVLA seeds solved

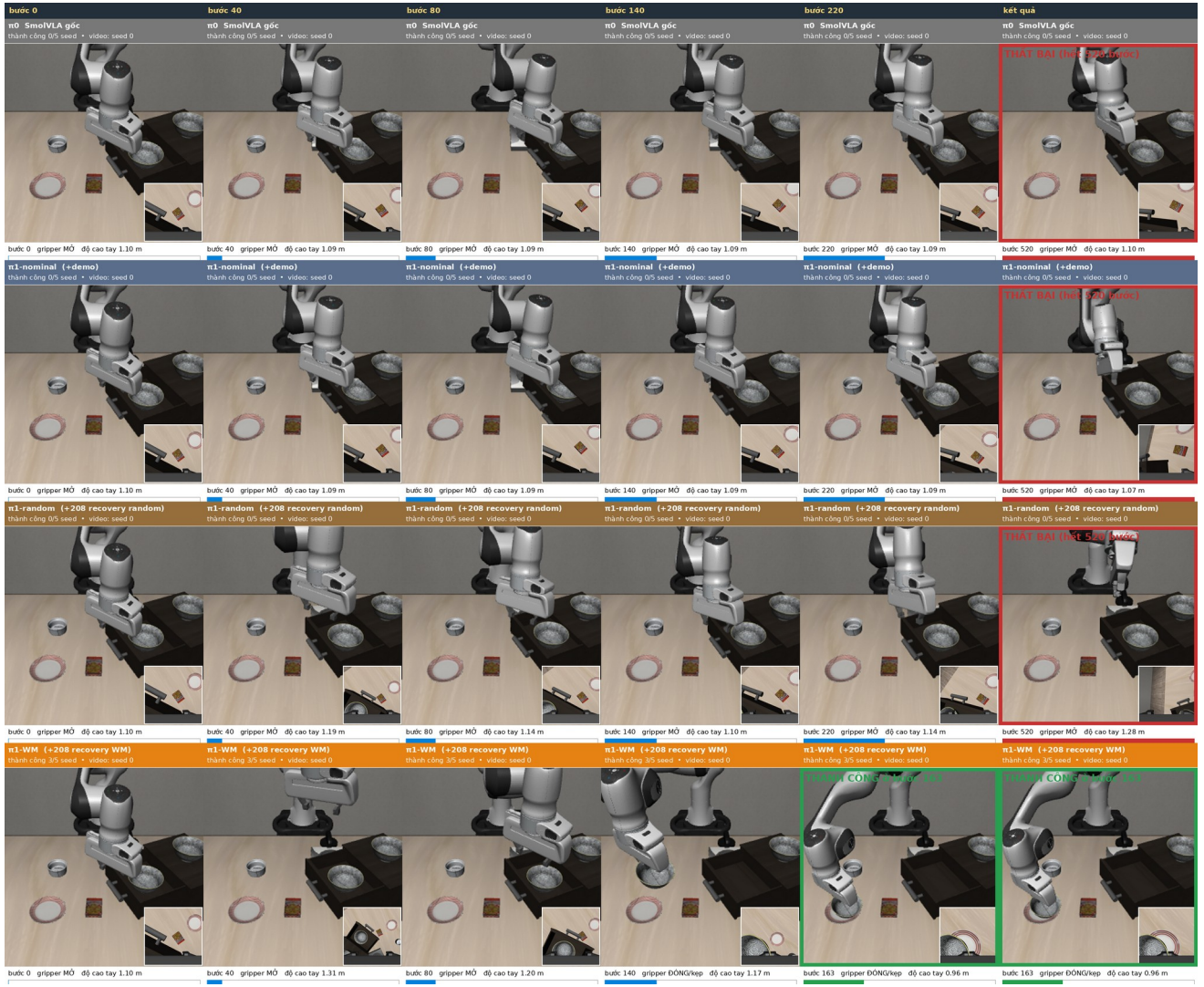
0/5 π1-nominal seeds solved

0/5 π1-random seeds solved

3/5 π1-WM (RetryVLA) seeds solved

π1-WM = RetryVLA pi1, seed-0 checkpoint (test RSR 0.439; 3-seed mean 0.418 ± 0.021, nominal 0.785)

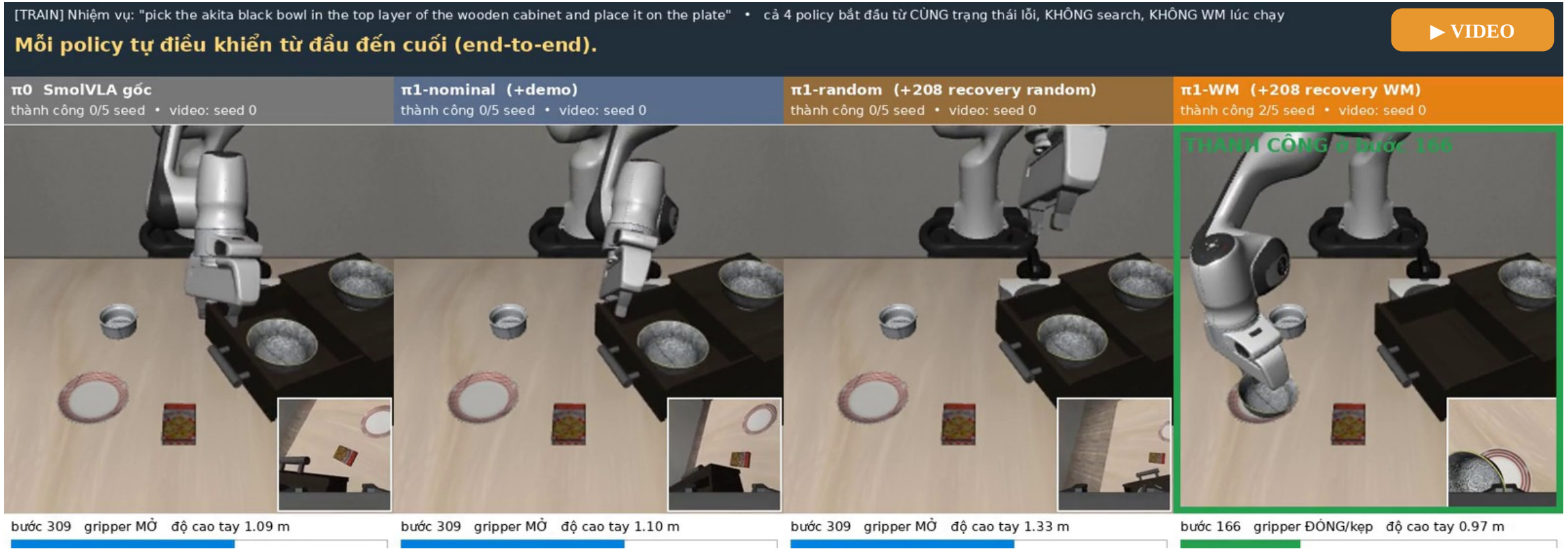
The VLA learned the WM's move: get clear, then retry



Same unseen drawer failure, seed 0 for every row

- rows: π0, π1-nominal, π1-random, π1-WM
- π0 / π1-nominal hover at 1.09-1.10 m for all 520 steps
- π1-random moves a little higher (1.14-1.24 m), still stuck
- π1-WM lifts itself to 1.31 m by step 40, comes back down, grasps at ~140, places at 163
- no world model or simulator at test time: the behaviour was distilled
- train-split episode (ep031): 0/5, 0/5, 0/5, 2/5
- π1-WM = RetryVLA pi1 seed 0 (RSR 0.439); 3 seeds 0.418 ± 0.021

Video: end-to-end on the training-split drawer failure



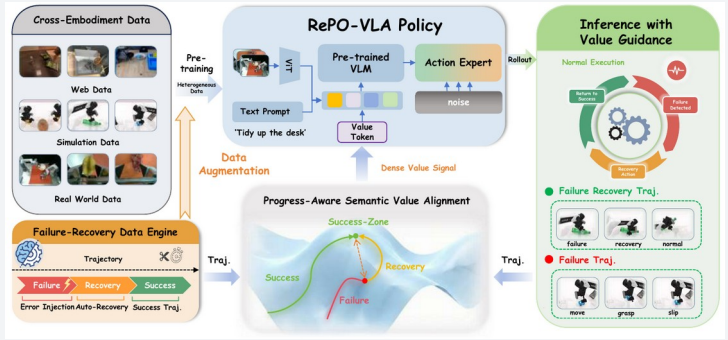
The same drawer failure as the A/B/C explainer (ep031 open6). π0 0/5, π1-nominal 0/5, π1-random 0/5, π1-WM 2/5 seeds. Still far from reliable on this failure. π1-WM = RetryVLA pi1, seed 0 (3-seed test RSR 0.418 ± 0.021); the plan now verifies bridges under ≥ 2 of 3 continuation seeds.

Related work: where RetryVLA sits

method	recovery	world model	objective / value	policy update	verification	extra at test time
RetryVLA (ours)	yes	latent DINOv2 (filter)	**competence C^pi of the frozen pi**	verified SFT + 50% nominal	sim, pi continues	**none**
RePO-VLA	yes	no	progress-aware semantic value	value-conditioned SFT	planner / human data	none (v = 1 token)
Dream2Fix	predicts fix	video WM makes failures	learned verifier filter	SFT of a VLM	learned verifier	VLM + correction
ReCoVLA	yes	no	VLM-compiled reward	residual PPO, VLA frozen	sim training	VLM + residual policy
RACER	yes	no	none (rich language)	SFT on perturbed demos	sim data generation	VLM supervisor
Neuro-symbolic	yes	scene-graph model	distance to sub-goal	none (re-plan)	no	detector + search
DINO-WM	no	latent DINOv2	distance to goal image	none (MPC / CEM)	no	planning each step
TD-MPC2	no	implicit latent	reward + TD value	online RL	online interaction	MPPI planning
RISE	indirect	video WM + value	progress value	RL in imagination	none	none
WMPO	emergent	pixel video WM	binary success reward	GRPO in the WM	few real rollouts	none
MimicGen	no	no	task success on replay	BC on generated data	sim replay	none

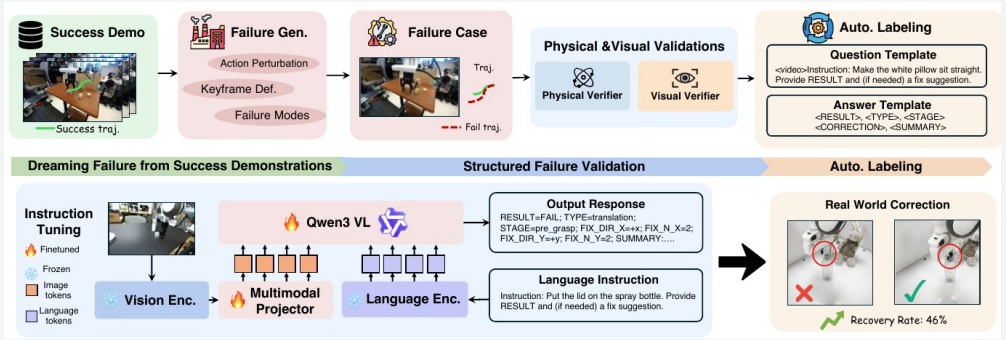
Different signal (competence of the frozen policy, not goal / progress / reward), different update (verified SFT, not RL), cheap test time (no VLM, no planning). Our WM is a filter; verification sets the labels.

Related work: recovery and failure correction



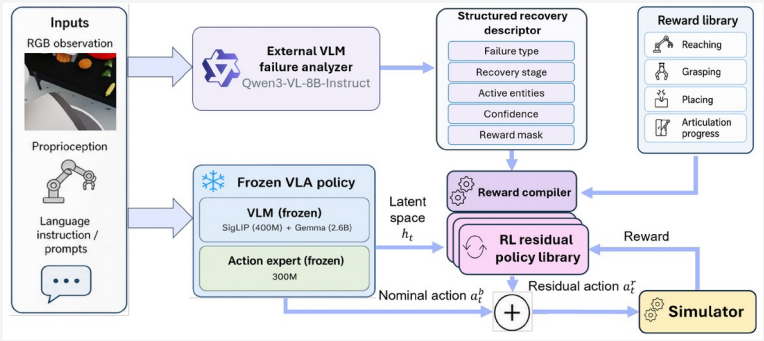
RePO-VLA (arXiv 2026)

progress value + value-conditioned SFT; recovery data from planner / humans. We search for it.



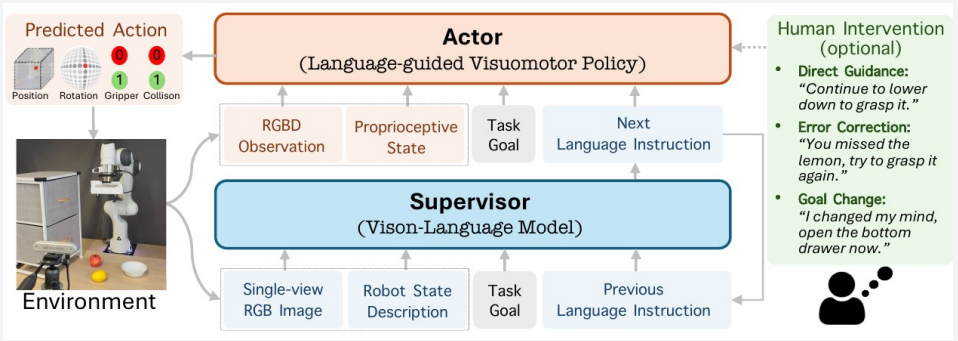
Dream2Fix (arXiv 2026)

video WM dreams failures; a VLM predicts corrections. Not distilled, verifier is learned.



ReCoVLA (arXiv 2026)

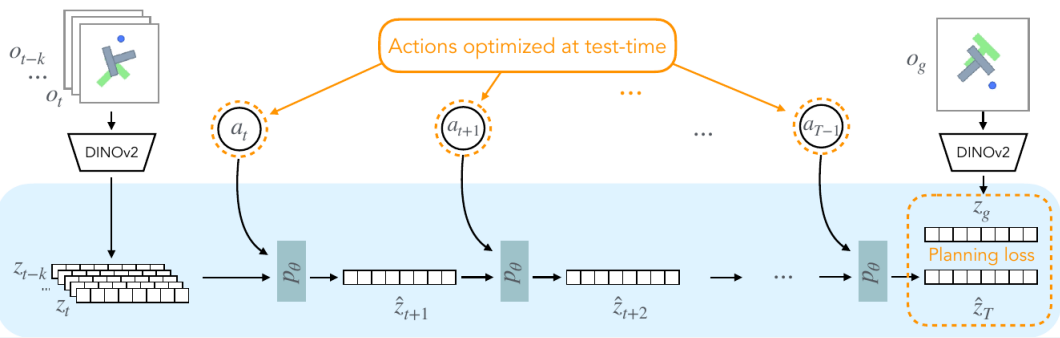
VLM-compiled reward + residual PPO on a frozen VLA. Ours: no reward, no residual at test.



RACER (ICRA 2025)

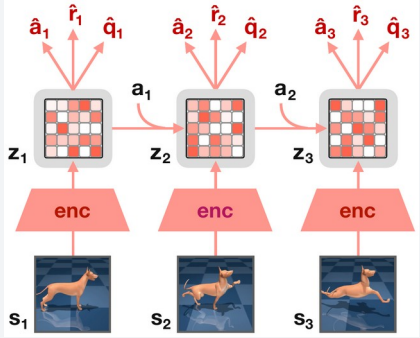
perturbed expert demos + language supervisor every step. Ignores whether pi can continue.

Related work: world-model planning and WM-assisted VLAs



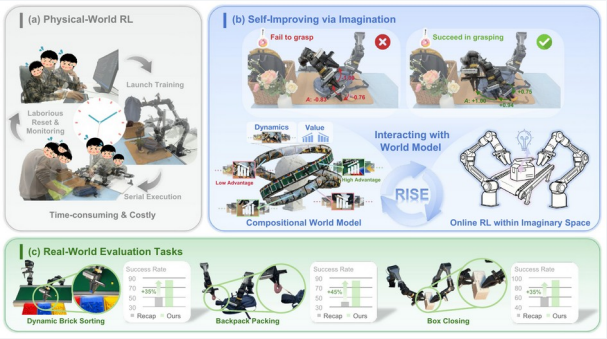
DINO-WM (ICML 2025)

our WM architecture; plans to a goal image. Goal-latent search: 0.240 vs C^pi 0.480 for us.



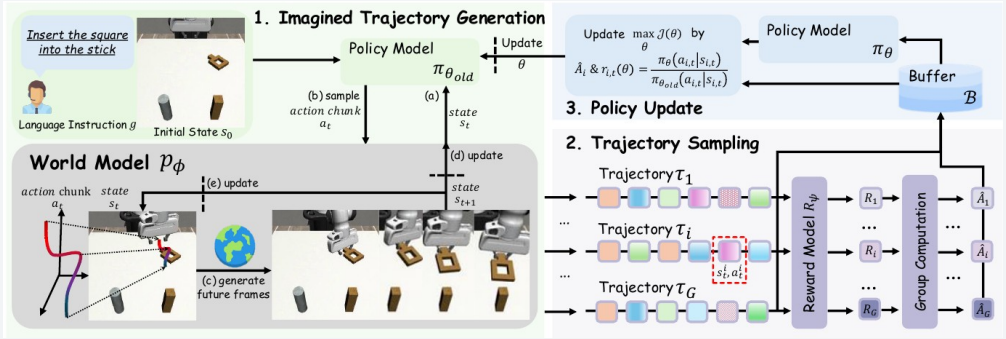
TD-MPC2 (ICLR 2024)

short latent plan + value at the horizon: our structure, but RL value and planning at test.



RISE (RSS 2026)

RL in imagination with a progress value. Our RISE-inspired arm: 0.068-0.113 < pi0.



WMPO (ICLR 2026)

GRPO inside a pixel video WM, whole task. Ours: short bridges, verified SFT.

Limitations and honest negatives

Method

- WM is mostly a filter over the proposal library (VLA samples + primitives): no library −0.175; planning from noise 0.398 vs 0.458
- WM data is not better than random-search data at equal count on Object (0.354 vs 0.362): only an efficiency claim holds
- self-improvement loop (M6) gave no gain beyond extra training; weak tasks (stove, cookie box) stay low
- one VLA backbone (SmolVLA, weak base 0.66 nominal) and one WM: no generality claim yet

Evidence and process

- many baselines are 1 seed (random, progress, RISE, RL, best-of-8, P3, all of Object): < ~4 pt is noise
- Object nominal 0.942 vs 0.944 is 1 seed: no eps-test yet
- Spatial test is a dev set from M7 on (used to choose M7 / M8); clean numbers only from Object
- infrastructure cost: ≥ 6 quota (EDQUOT) incidents, lost tmux servers, MuJoCo segfaults, OOM: several SFTs retrained (code now patched)

Where we stand

Supported

- competence-guided search + verified distillation: RSR 0.118 → 0.418 ± 0.021, nominal within 2 pt (3 seeds)
- competence ≫ goal / progress in search and SFT; search result replicated on held-out Object
- distillation ≫ demo-only, best-of-N, imagination-only and residual RL
- B4 question answered: the value is in pi0 continuation labels
- WM: 2-3× more verified recoveries per sim; matches a real-dynamics prefix at ~1/3 transitions

Open

- the WM does not yet plan bridges without the proposal library (M11)
- acquisition efficiency → better policy at matched sim budget? (P4, not run)
- end-to-end cost accounting incl. WM / critic training (P5, partial)
- 1-seed baselines and Object; eps-test on Object nominal
- one VLA, one WM, two suites; LIBERO-Goal / LIBERO-10 not run

Next steps: PoC plan status (PLAN.md)

#	experiment	question	status
P1	WM as planner (M11): no library, broad-noise CEM, WM v2	can the WM plan bridges itself?	**done: no, WM = filter**
P2	progress / task-success critic baselines (B2-B4)	policy grounding or any value?	**done: policy grounding**
P3	recovery-nominal Pareto + L_preserve	M-B constraint, forgetting	**done: r 0.5, no L_preserve**
P4	matched-sim-budget distillation on Object, 3 seeds	does efficiency give a better policy?	**todo**
P5	end-to-end cost accounting (WM + critic amortized)	efficiency claim	**partial**

1 C^pi v3

B4-style trainer on the Cpi-pool pi0 labels (+0.064 solved over v2), after a held-out check

2 More seeds

3 seeds for key 1-seed baselines and for Object (nominal eps-test)

3 Suites

LIBERO-Goal and LIBERO-10 with the frozen recipe; L1-L4 breakdown

4 Real world

adaptation not decided

Long-term plan

PoC	**Now**	close out: P4 on Object (3 seeds), P5 cost accounting, more seeds, C^pi v3
M-A	**Next**	generality of the planner: a second VLA backbone and / or a second (pixel / video) WM; claim 'general' only after
Suites	**Then**	LIBERO-Goal and LIBERO-10 with the frozen recipe (Setting A), LIBERO-Recover L1-L4 breakdown
Loop	**Later**	pi_k → C^{pi_k} → search → verified distillation → pi_{k+1}, critic re-grounded on pi_k each round
Analysis	**Open**	action coverage of verified bridges vs pi0 proposals; learned proposal head only if coverage is the bottleneck; real world not decided

Next decisions

1. adopt C^pi v3 (Cpi-pool labels, B4 trainer) for all future search, after a held-out check?
2. run P4 on Object with 3 seeds before scaling to LIBERO-Goal / LIBERO-10?
3. WM as planner: invest further, or report the WM honestly as an efficient filter?
4. real-world adaptation: not decided

Appendix: M0-M5 details

the earlier progress slides, kept as reference · single seed, Spatial split · current numbers are in the main part

M0: SmolVLA is decent at nominal tasks, bad at recovery

0.66

nominal LIBERO-Spatial success (500 ep)
lerobot-eval cross-check: 0.68

0.12

recovery success (RSR) on 56 held-out failure scenes, 560 ep

5 / 160

drawer-task failures recovered



Training data SmolVLA learned from: a nominal LIBERO-Spatial demo (agent + wrist camera, action per step)

- checkpoint is weaker than the ~0.90 reported for SmolVLA, but our engine matches lerobot-eval, so the interface is correct
- fine-tuning on the Spatial demos alone reaches 0.80 nominal. That checkpoint (pi1-nominal) becomes the control and the stronger base
- replanning every step (n_action_steps=1) gives +5 points at 10x the cost, so we keep 10

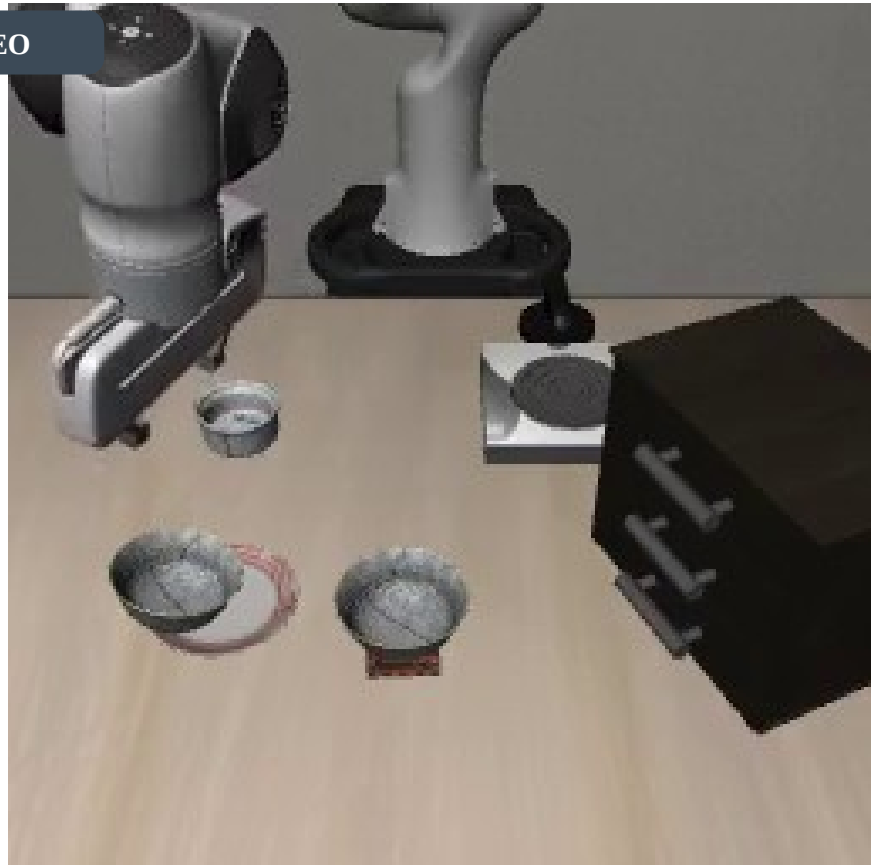
Video: SmolVLA on held-out failure states

VIDEO



Drawer failure: pi0 never recovers (26 s)

VIDEO



Ramekin failure: pi0 keeps trying the same grasp (26 s)

Official protocol, 520 steps. These are the 88% of test episodes we want to fix.

M1: how one recovery candidate is tried

Restore

load the exact failure snapshot

Segment (50 steps)

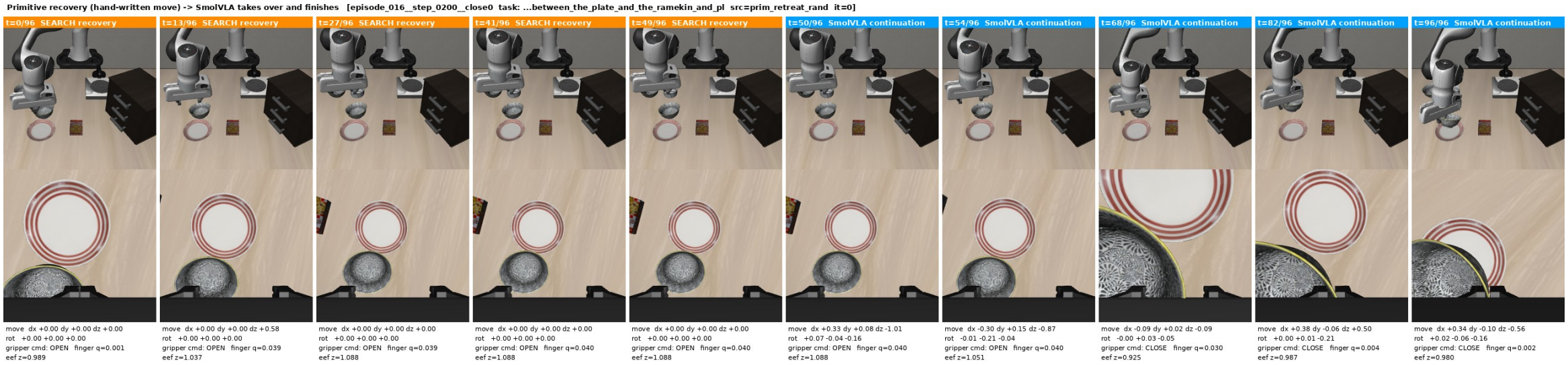
open-loop actions from search: SmolVLA samples, smooth perturbations, primitives

SmolVLA continues

frozen pi0 drives closed-loop up to 300 steps

Judge

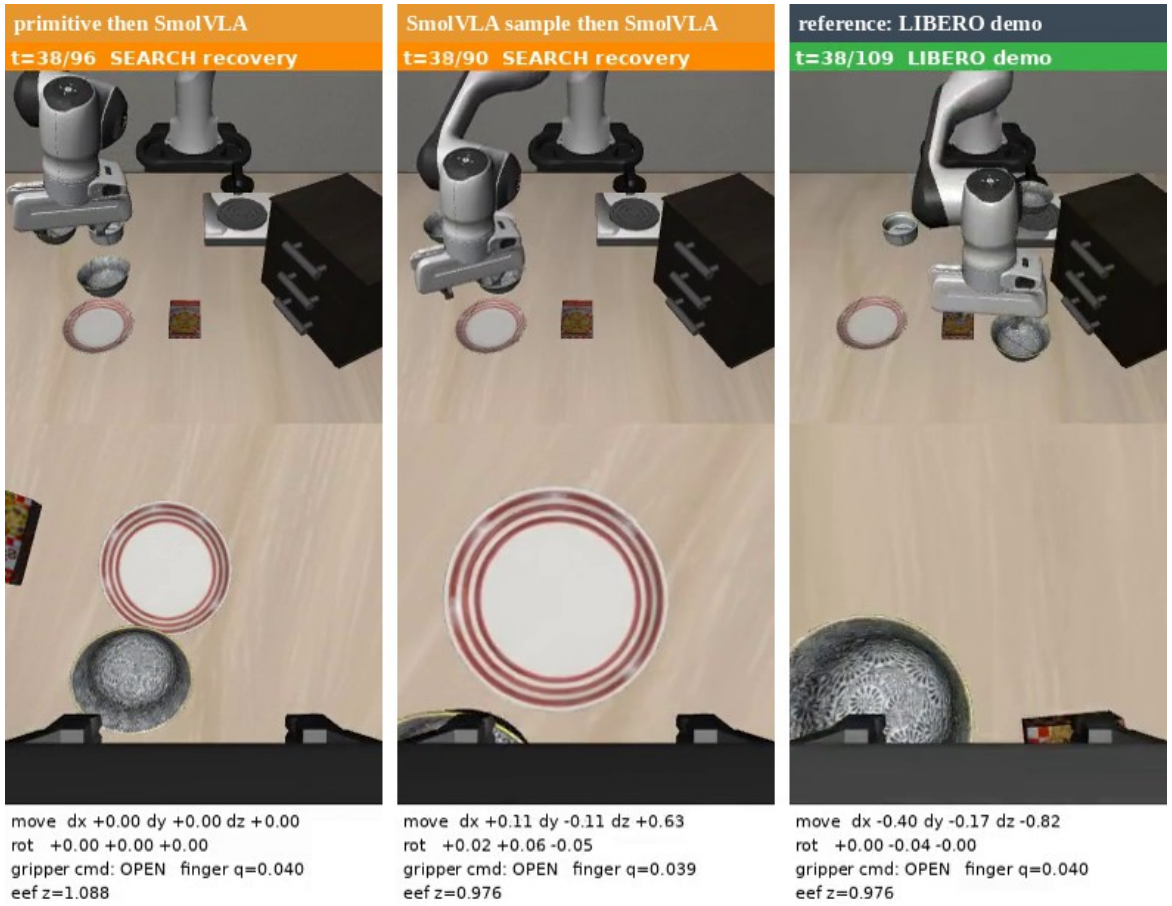
simulator checks the BDDL goal: bowl on plate?



Orange = searched segment (here a hand-written 'open + lift' primitive). Blue = SmolVLA takes over and places the bowl. Competence-targeted: the segment does not solve the task, it returns the robot to a state SmolVLA can finish from.

Video: oracle-search recoveries

VIDEO



Orange bar = searched 50-step segment, blue = SmolVLA continuing, green = nominal demo. Overlay: step, segment, action given to the policy, finger opening.

M1: search = propose, score in the simulator, refine

1

Propose 64

24 SmolVLA samples (different noise), 24 smooth perturbations, 16 primitives

2

Score

simulate each: success × speed + small goal-progress shaping − jerk

3

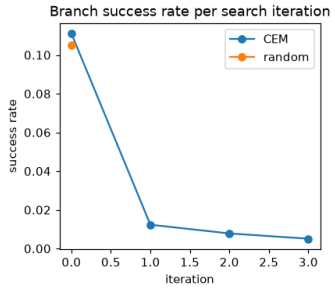
Keep elites

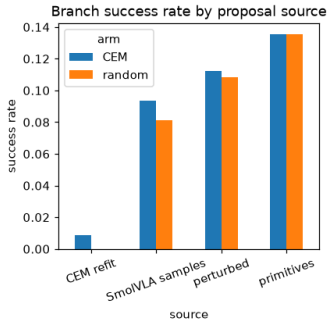
8 best candidates

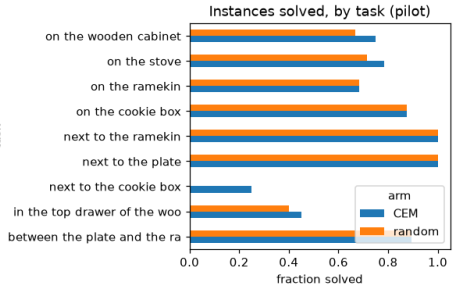
4

Refine

CEM: sample around elites, up to 4 rounds (≤256 sims / instance)



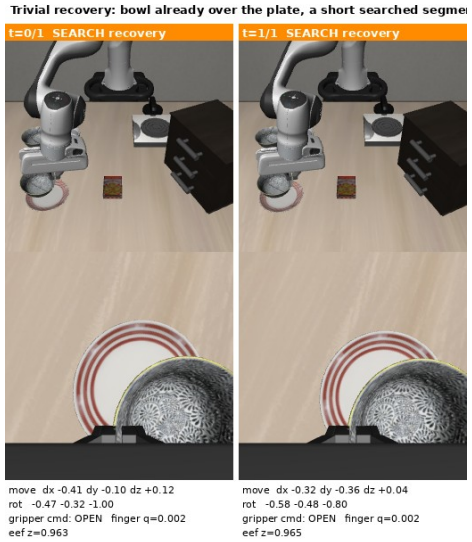




What we learned

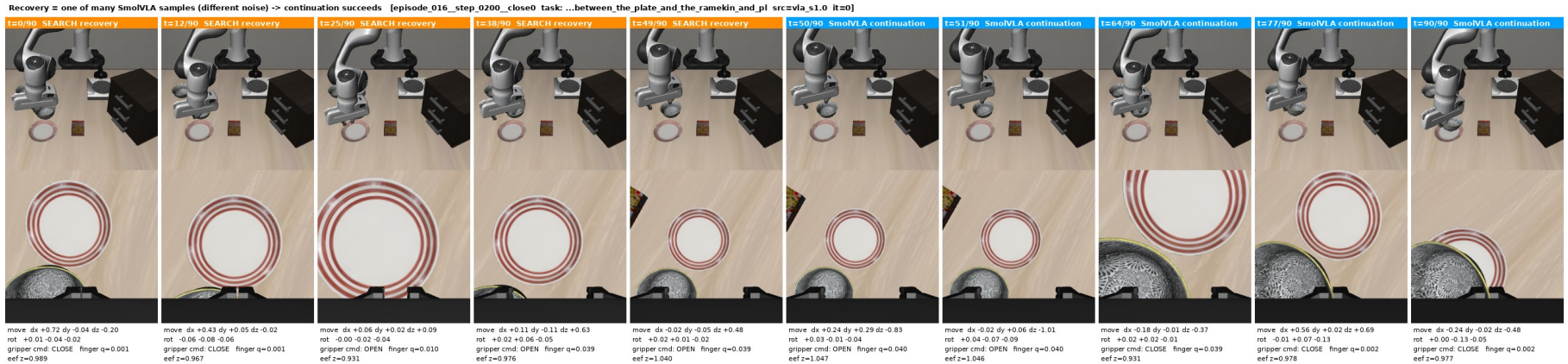
- ~70% of pilot failures have a recovery reachable in 50 steps
- the first CEM was broken: averaging different strategies into one Gaussian drops success 11% → 1%
- fix: refine around ONE elite at a time. On hard instances it then solves 68% vs 55% for random at equal budget
- ~1/3 of recoveries are trivial (bowl already over plate: just open)

What the recovery training data looks like

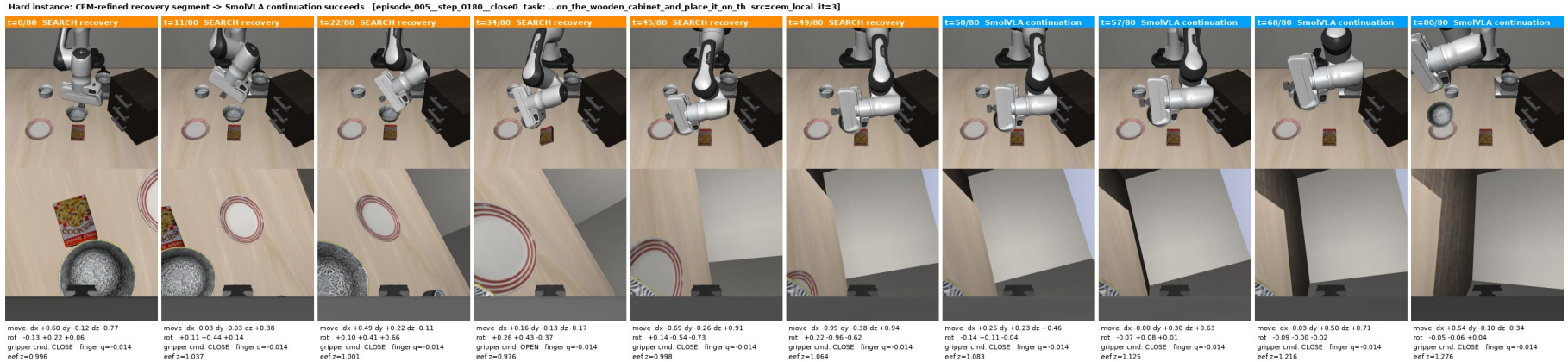


Trivial

2 steps: release the bowl already above the plate. Removing these HURT RSR (0.254 → 0.220)



One of 24 SmolVLA samples (different noise) happens to open and back off; continuation succeeds



Old CEM on a hard instance: jerky, gripper chatter (3-4 flips / 50 steps vs 0.9 in demos). Searched segments are 4-6x jerkier than demos

Video: why random-noise search proves little

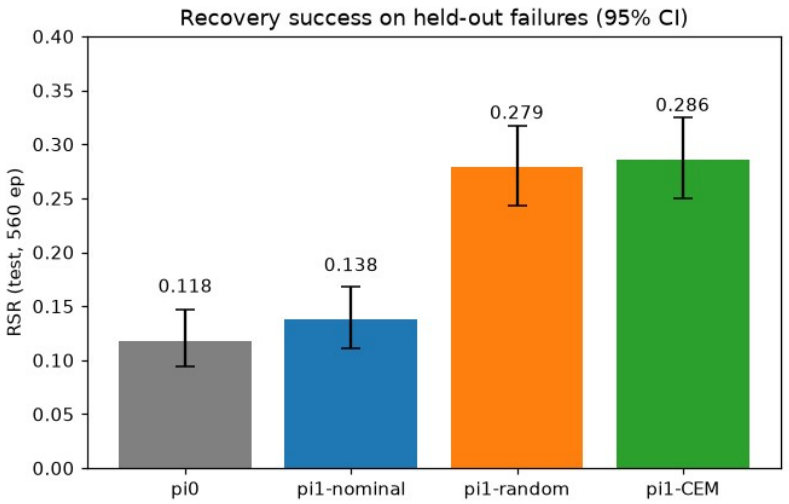
VIDEO

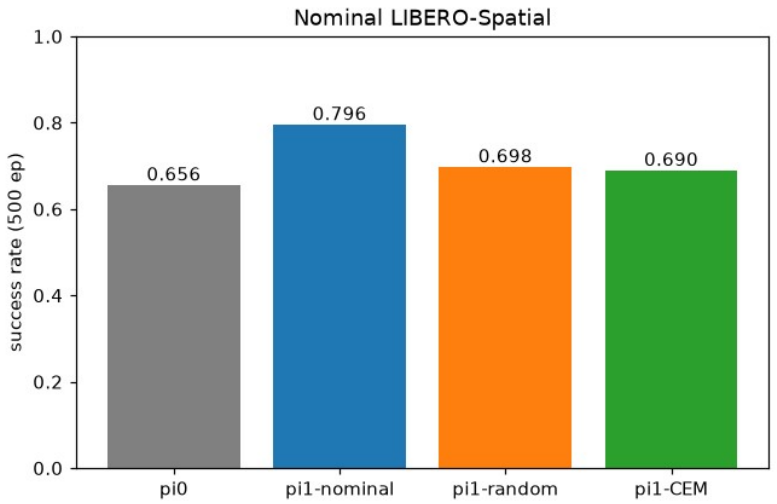


Old CEM on a hard instance

- arm thrashes, gripper flips open/closed with no effect on the fingers
- it still 'succeeds' afterwards, so it lands in the training set
- 4-6× jerkier than demos: a likely reason recovery SFT costs nominal success
- motivates: smoothness cost, elite-local refinement, and scoring by a model of what SmolVLA can finish

Distilling simulator-verified recoveries works





SFT arm	recovery data	test RSR	nominal
pi0	-	0.118	0.656
pi1-nominal (control)	demos only	0.138	0.796
pi1-random	241 random-search	0.279	0.698
pi1-CEM	241 CEM	0.286	0.690
ratio 0.25	241 random	0.254	0.726
ratio 0.25, no trivial	170 random	0.220	0.766

2.4×

test RSR vs pi0 (0.118 → 0.28), +14 pts over the fine-tune-only control

- CEM ≈ random here: the old CEM's useful data all came from its random first round
- cost: nominal 0.80 → 0.70 at a 50% mix. Trade-off set by the ratio
- trivial recoveries are useful supervision

Replacing the simulator with a world model

M2: latent world model + competence critic · M3: search in imagination

Why a world model if we already have a simulator?

Oracle search (M1)

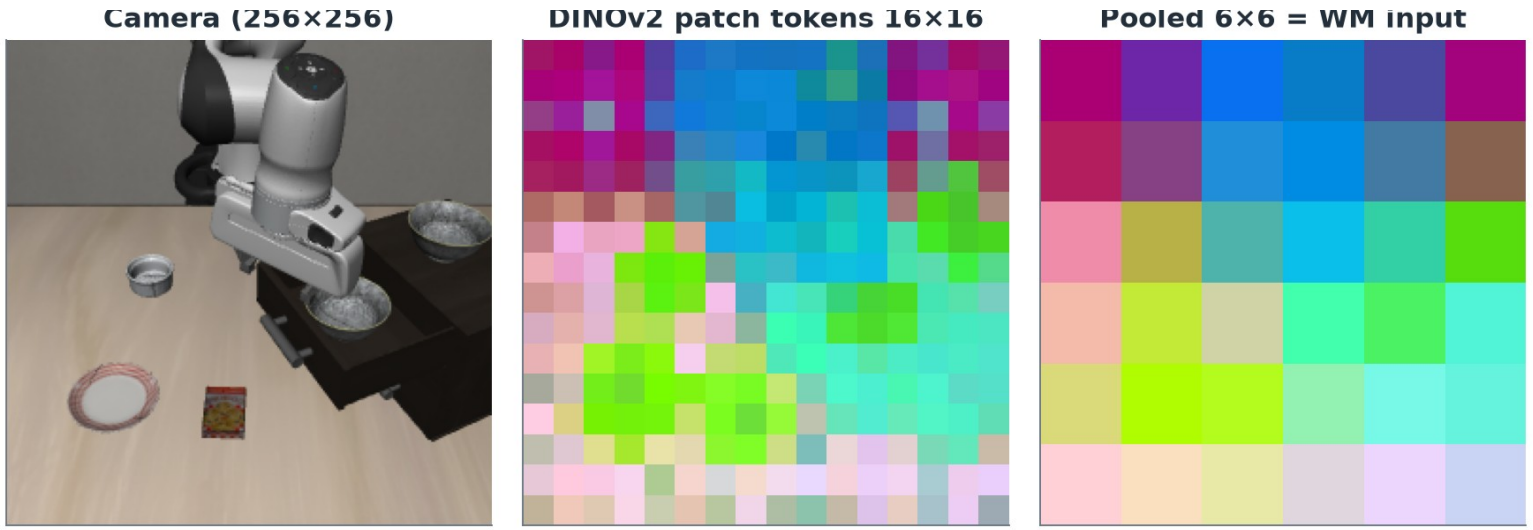
- scores a candidate by running it for real
- up to 256 simulator rollouts per failure
- exact, but only exists where a simulator of THIS scene exists
- upper bound on what search can find

World-model search (M3)

- scores a candidate by imagining it in latent space
- ~800 candidates in a few GPU seconds
- simulator only to verify the top 16 (later: real robot, or none)
- needs: accurate dynamics on failure states + a critic that knows what SmolVLA can finish

Claim to test: at a fixed number of real rollouts, WM-guided search finds more recoveries than proposing at random.

State = frozen DINOv2 patch tokens + robot state



Same drawer failure state. Token colors = first 3 PCA components: bowl, drawer, arm and table separate without any training

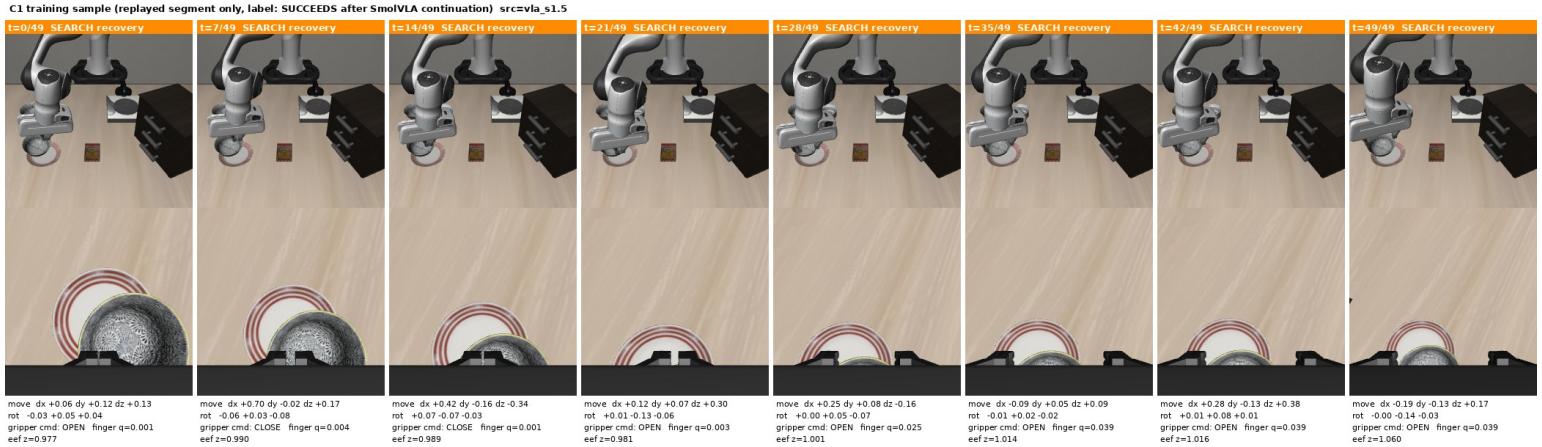
C1 training data for the WM and critic

- 144 TRAIN failure instances × 32 candidate segments = 4,608 simulated trajectories
- latents every 5 env steps + the actions + outcome label (did SmolVLA finish afterwards?)
- the failure episodes containing the 53 hard search targets are excluded: those failures stay UNSEEN

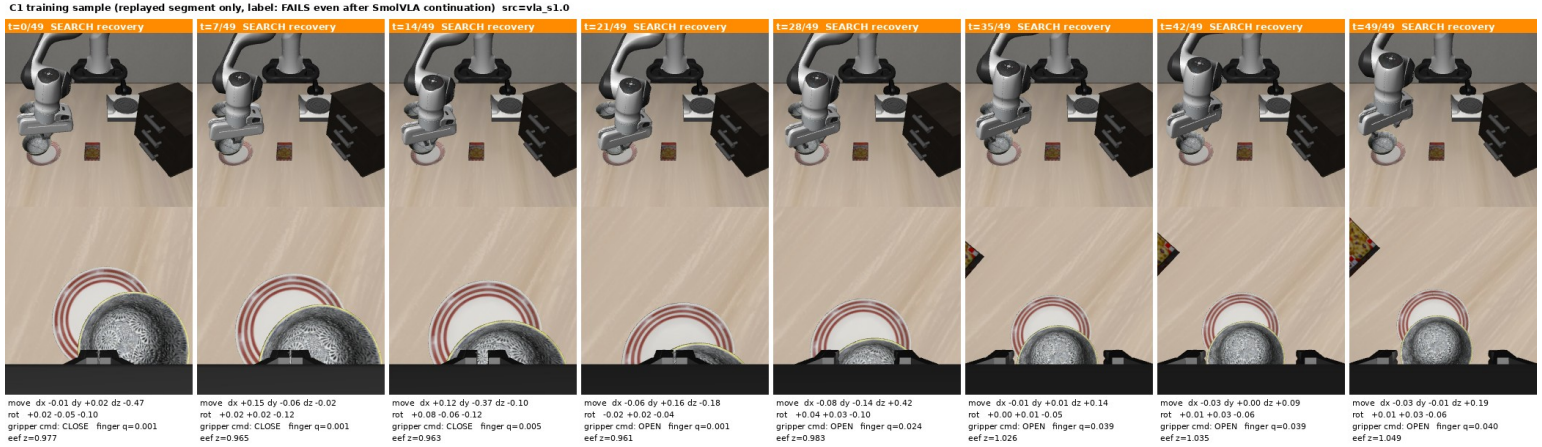
h_t (per time step)

- agent cam 6×6 tokens
- wrist cam 6×6 tokens
- robot state q (8-D): eef pos, axis-angle, finger
- = 72 tokens × 384-D + 1 state token
- spatial tokens, not one global vector: L3/L4 failures depend on where things are

One failure state, two candidates, two labels

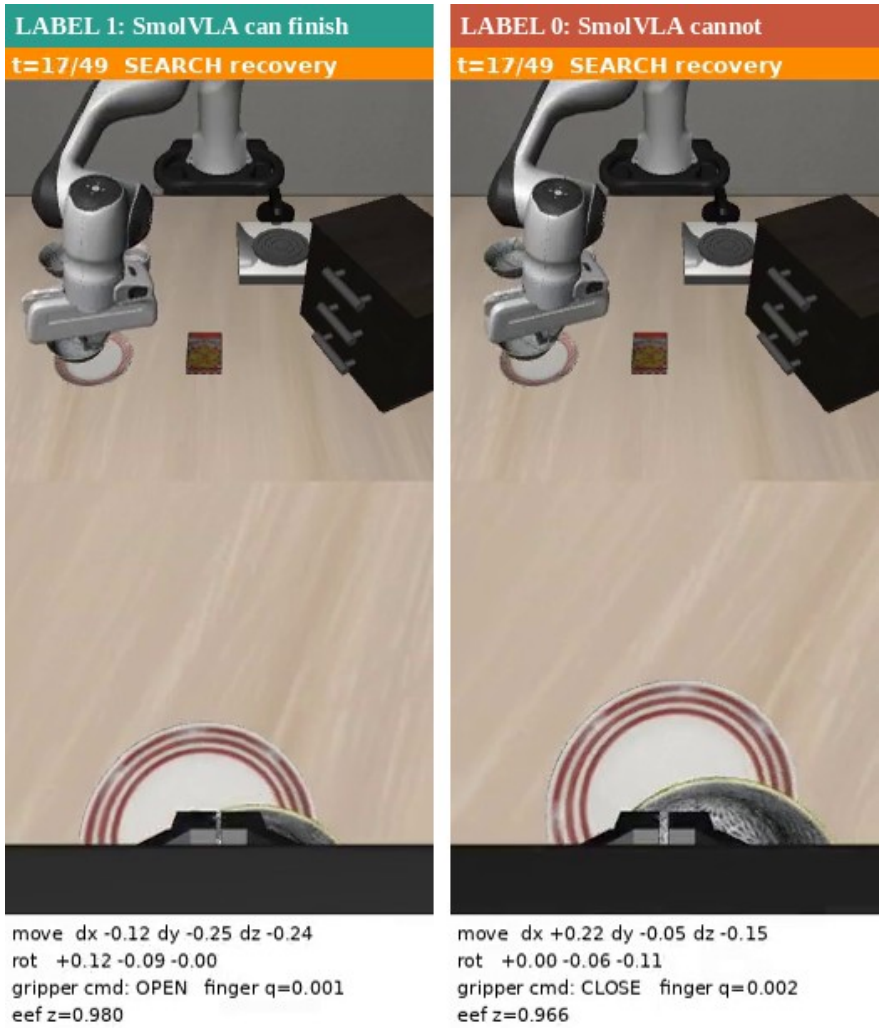


Label 1: opens at t=21 and lifts. The bowl stays on the plate and SmolVLA finishes



Label 0: also opens, but drags the bowl off the plate (t≥35), so SmolVLA cannot finish. The critic must tell these apart from latents alone

Video: WM data, same state, opposite labels



Same failure state, two candidate segments

- left: opens and lifts, bowl stays on the plate, so SmolVLA can finish (label 1)
- right: opens but drags the bowl off the plate, so SmolVLA cannot (label 0)
- 4,608 such segments teach the WM how latents move and the critic which end states are recoverable
- replayed from the stored actions: bit-exact

M2: latent dynamics h(t+1) = h(t) + F(h(t), 5 actions)

h_t

72 patch + 1 state token

+ action token

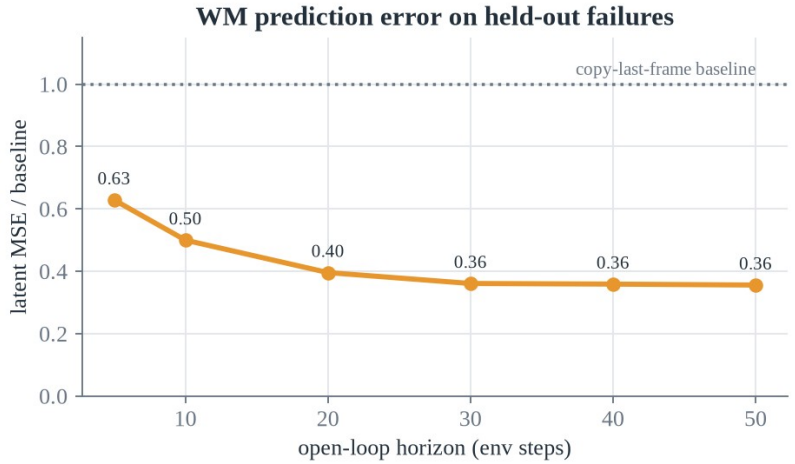
MLP(5 × 7-D actions)

Transformer

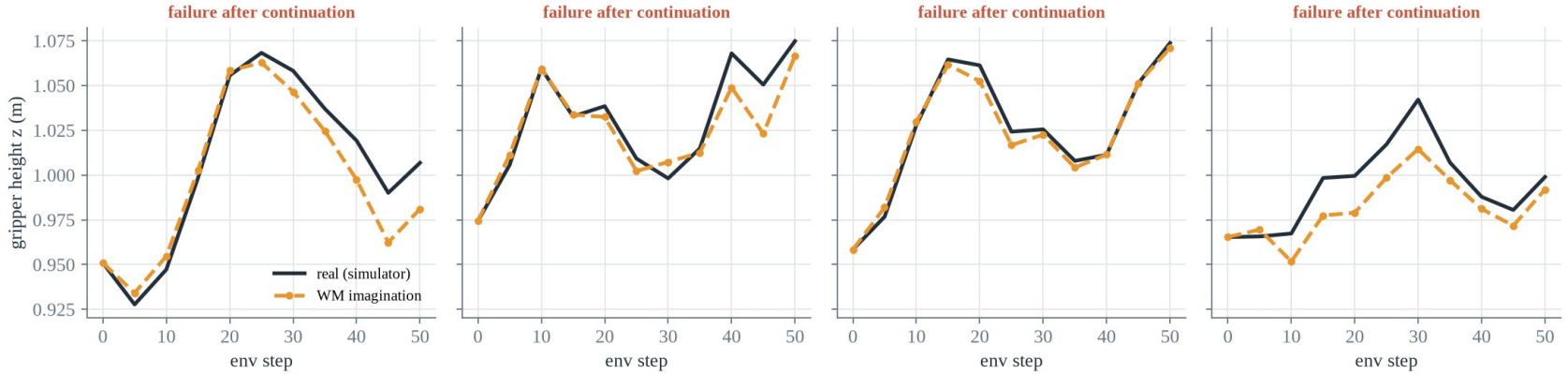
6 layers, width 384

4 heads

ensemble Δh: mean = prediction, spread = uncertainty



- loss = 1-step teacher-forced + 10-step open-loop rollout (search needs 50-step imagination)
- 11.6M params, 20k steps, ~1.5 h on one A100

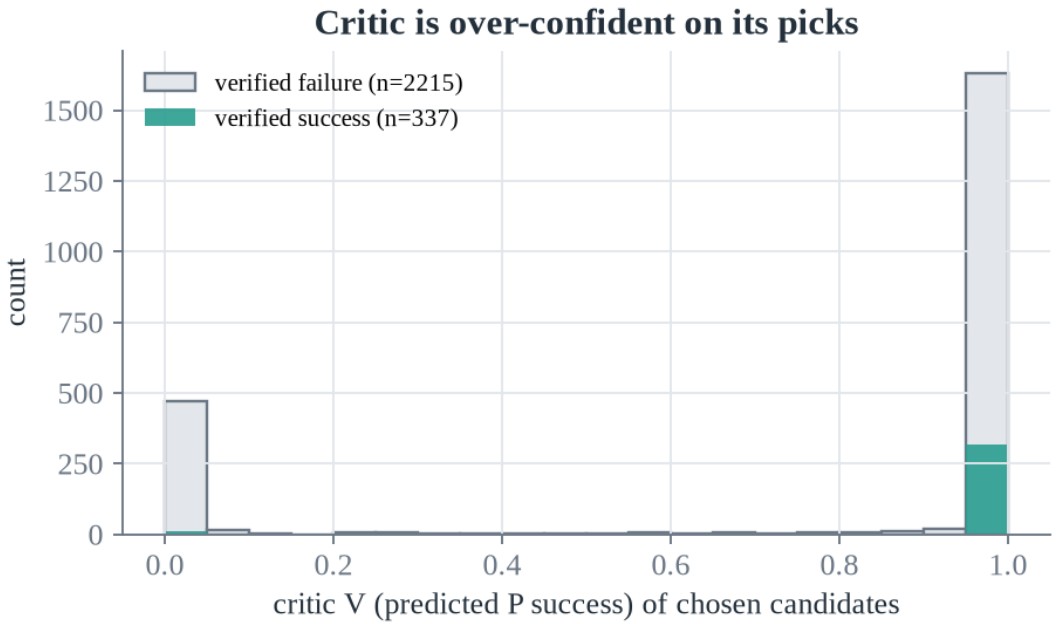


Held-out failures: gripper height, real (black) vs imagined from the start frame + actions only (amber). Error is 0.36-0.63× the copy-last-frame baseline

M2: the competence critic V(z, q, task)

"From this (possibly imagined) state, will frozen SmolVLA finish the task?"

- small transformer over the same tokens + learned task embedding
- trained on real AND WM-imagined end states (50/50), so it is calibrated for what search feeds it
- competence-targeted: lifting out of a drawer scores high even though the bowl gets no closer to the plate
- held-out AUC 0.71 real / 0.67 imagined; top-4 picks succeed 2.1× the base rate



Ranks well, calibrated badly

Almost every pick gets V≈1 while ~12% verify. Fine for ranking. Needs fixing before V can weight training (AWR).

M3: how the world model searches

1 Encode

real failure frame → DINOv2 latent z0, robot state q0

2 Propose 256

same proposal pool as M1: SmolVLA samples, perturbations, primitives

3 Imagine

WM rolls each 50-step candidate forward: z0 → z50 (10 steps of 5 actions)

4 Score

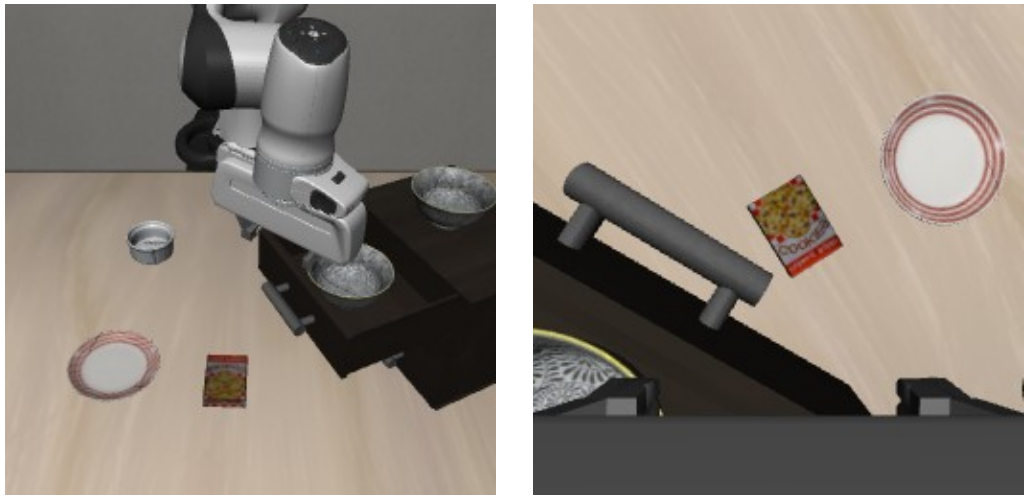
$S = V(z50) - \lambda \cdot \text{uncertainty} - \beta \cdot \text{jerkiness}$

5 Refine in imagination

2 rounds of elite-local CEM, ~800 candidates, zero simulator calls

6 Verify top-16

only these touch the simulator; successes become training data

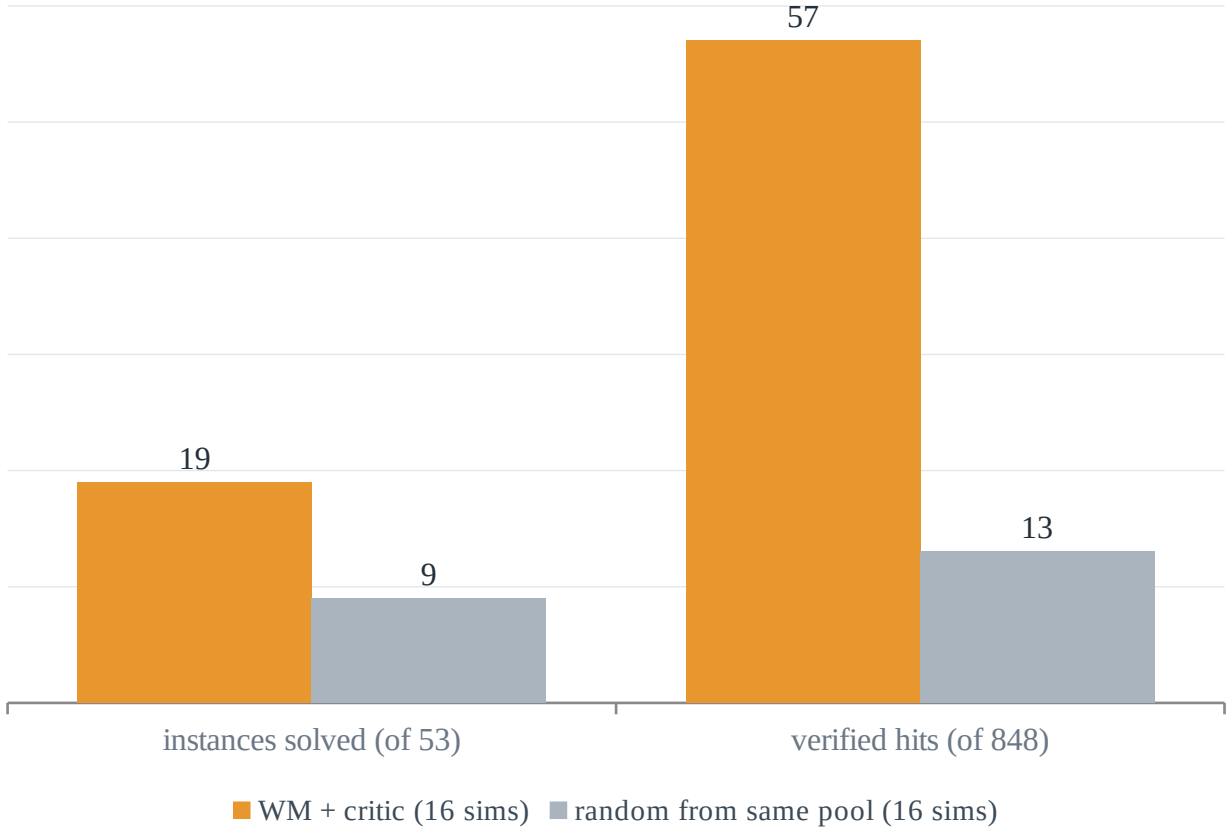


Input: one real failure observation (agent + wrist)

Fair comparison

- random arm: 16 candidates drawn from the same 256-proposal pool
- both arms: exactly 16 simulator rollouts per instance
- 53 hard instances whose failure episodes the WM/critic never saw

M3: 2× more recoveries on unseen hard failures



Picks are also smoother: jerk 0.017 vs 0.027

36% vs 17%

hard failures solved, same simulator budget (sign test p = 0.04)

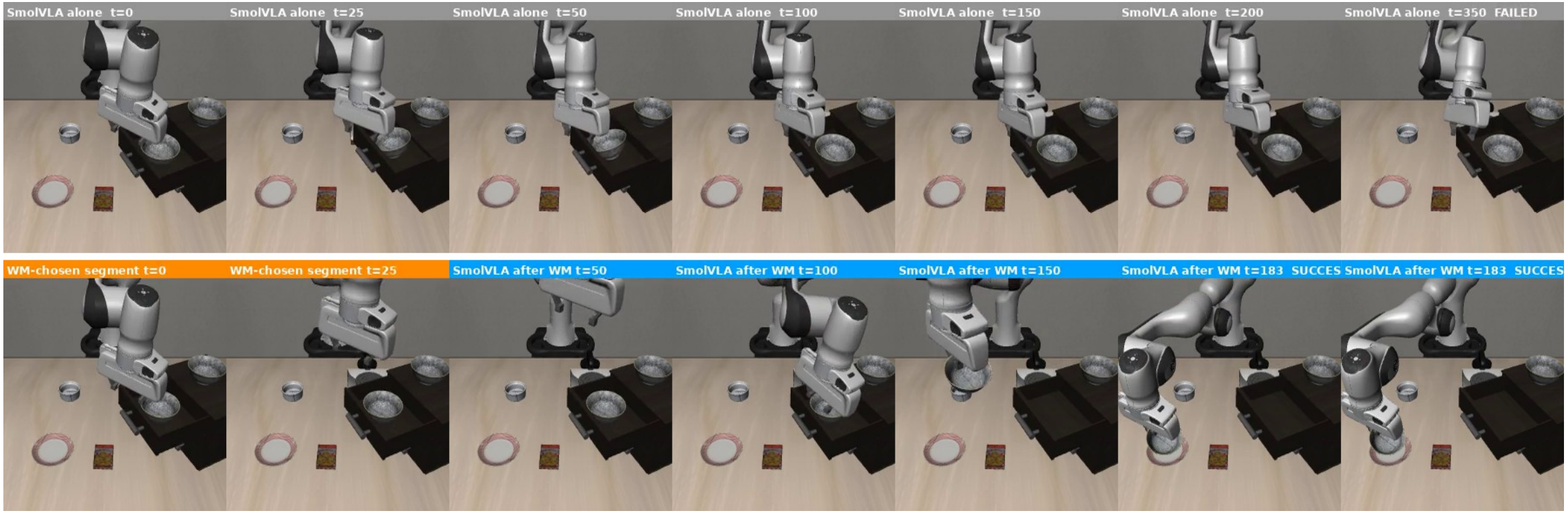
35 / 45

recoveries found by latent CEM: plans the WM invented, never simulated before verification

Caveat

The gain is concentrated in the drawer task (WM 12/20 vs random 1/20). Other tasks: 7/33 vs 8/33. N = 53, one seed.

Same failure state: SmolVLA alone vs WM-guided



Top: SmolVLA alone hovers at the drawer mouth for 350 steps. 0/3 seeds, 0/4 pre-checks, random 0/16.

Bottom: the WM picks 'lift straight up' (orange). From there SmolVLA reaches in, grasps and places the bowl. 3/16 verified.

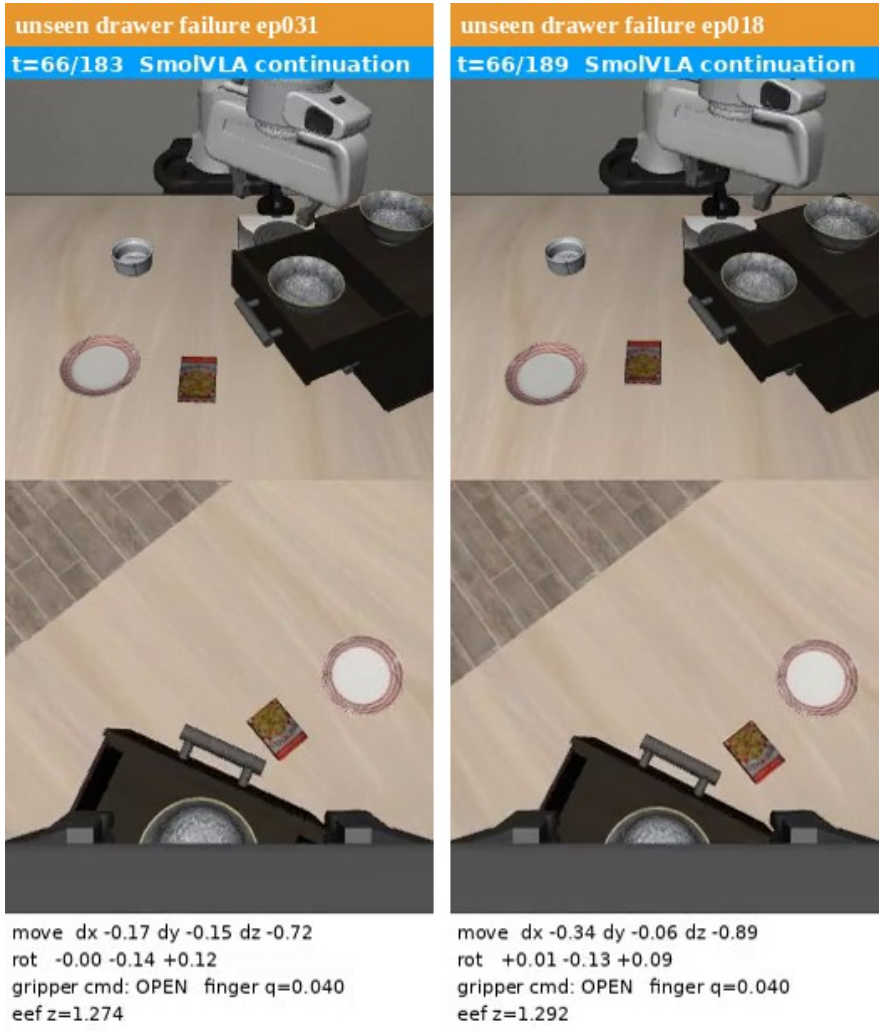
Video: SmolVLA alone vs WM-guided, same failure

VIDEO



Left: SmolVLA alone hovers at the drawer for 350 steps. Right: WM-chosen 50-step lift (orange) then SmolVLA grasps and places the bowl.

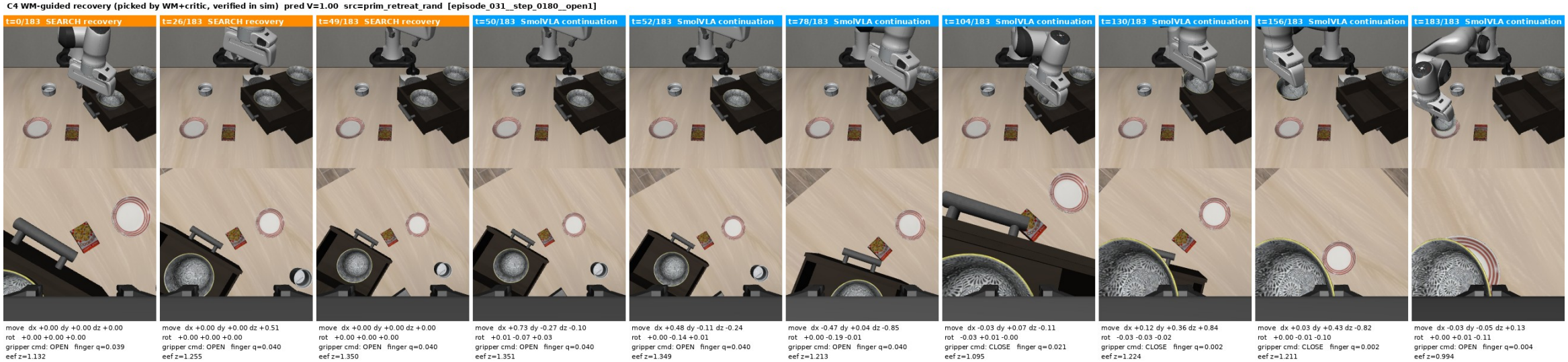
Video: WM-found recoveries on unseen failures

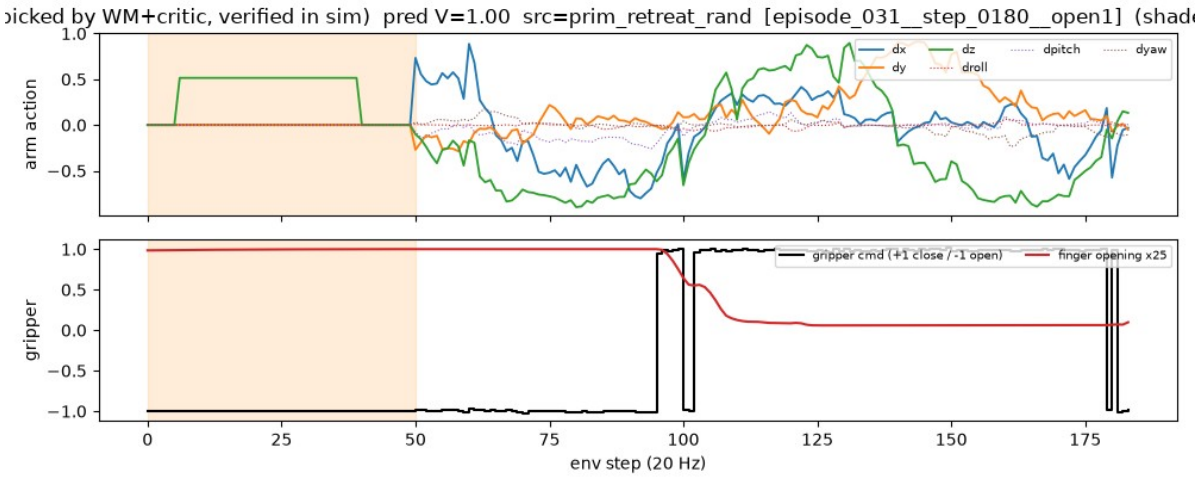


Picked by WM + critic, verified in sim

- orange: the 50 steps the WM chose in imagination (lift clear of the drawer)
- blue: SmolVLA reaches in, grasps, places the bowl
- neither failure episode was in the WM's training data
- same strategy found on different failures: the WM generalizes the 'get clear first' idea

What the WM chose, step by step





- shaded = the 50 steps the WM chose: one clean dz = +0.5 move, gripper open, no chatter
- white = SmolVLA's own actions afterwards; finger closes at t≈95 (red), bowl placed at t=183
- contrast with the jerky old-CEM segment in M1

M4: does WM-found data improve the VLA? Not yet measurable

policy (init)	recovery data	test RSR	drawer	nominal
pi1-random (pi0)	241 random	0.279	18/160	0.698
A (pi1-nominal)	241 random	0.293	24/160	0.756
B (pi1-nominal)	241 random + 45 WM	0.298	28/160	0.748

- stronger init (pi1-nominal) recovers nominal 0.70 → 0.75 and nudges RSR up
- +45 WM trajectories: +0.005 RSR, drawer 24 → 28. Paired episodes 61 vs 58: noise
- the dose is tiny: 45 WM trajectories ≈ 8% of training samples

So M5 asks it properly

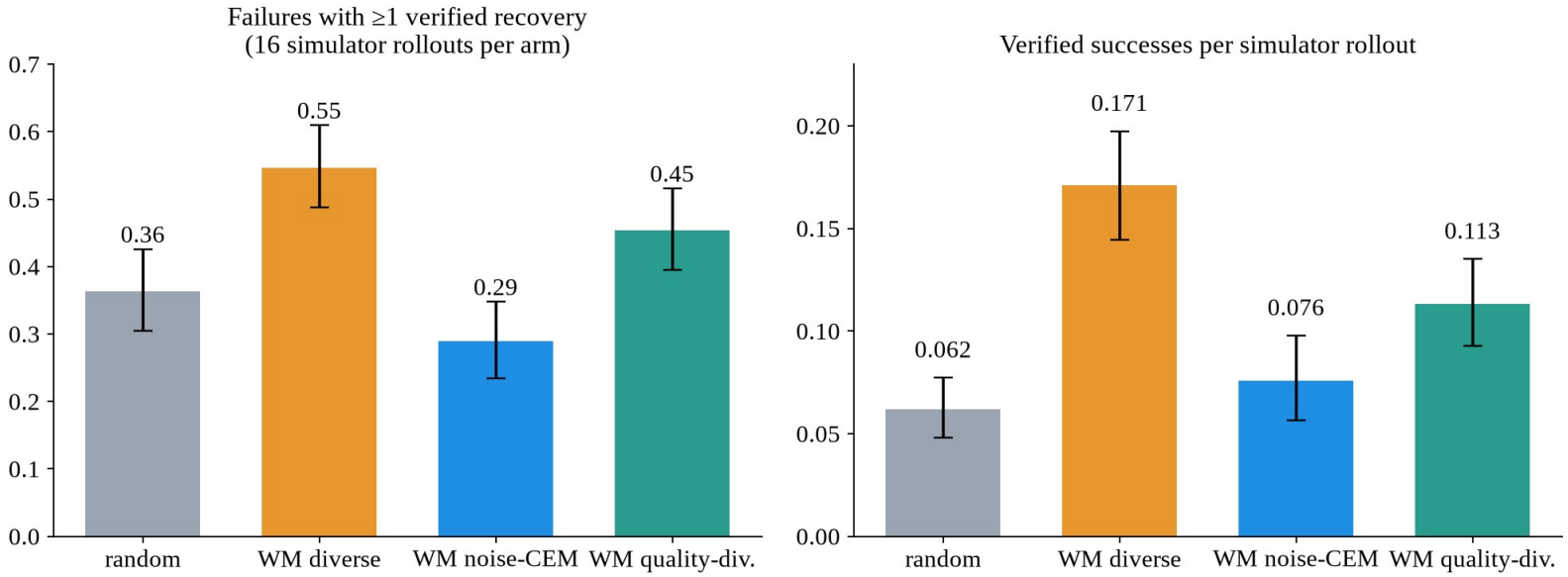
- generate WM data on all 108 pilot failures (not just 53 hard)
- same trajectory count WM vs random
- WM-only vs random-only SFT, same init, same recipe
- measure RSR, drawer, nominal

M5: WM data vs random data (Spatial, seed 0)

test RSR 0.25 (random data) → 0.44 (WM data, seed 0; 3 seeds: 0.418 ± 0.021), same #trajectories

on held-out Object the equal-count gap did not replicate (0.354 vs 0.362)

M5b: 4 search arms, 256 fresh failures, same 16 sims each



N = 256 paired pi0-failure instances (same instances, same continuation seeds, 95% bootstrap CI). WM diverse vs random: McNemar p = 4e-9

random

16 uniform picks from the proposal pool (no WM)

WM diverse

pool + latent CEM, k-means diverse top-16 (M5 default)

WM noise-CEM

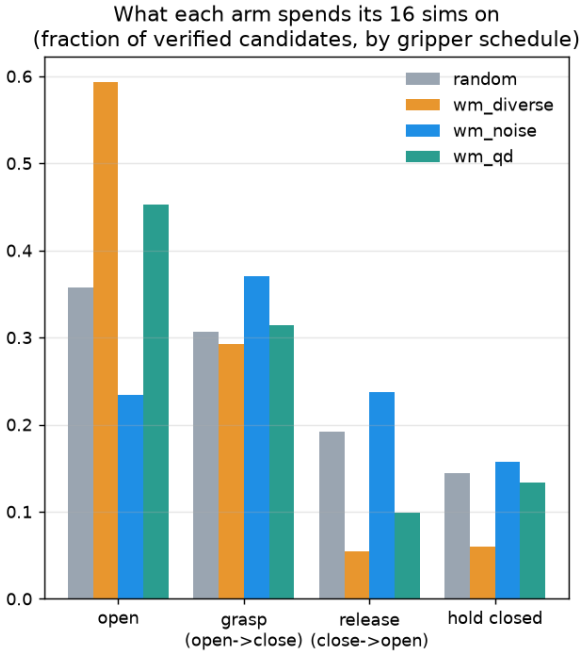
NEW: CEM in SmolVLA's flow-matching noise: on-manifold by construction

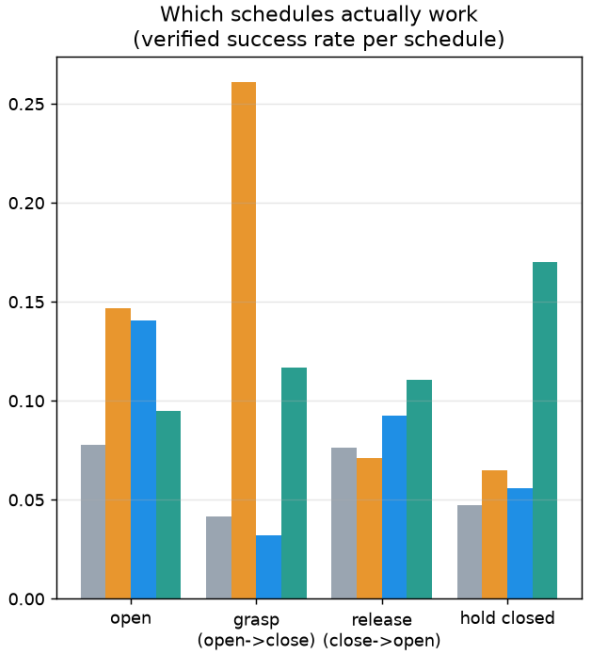
WM quality-div.

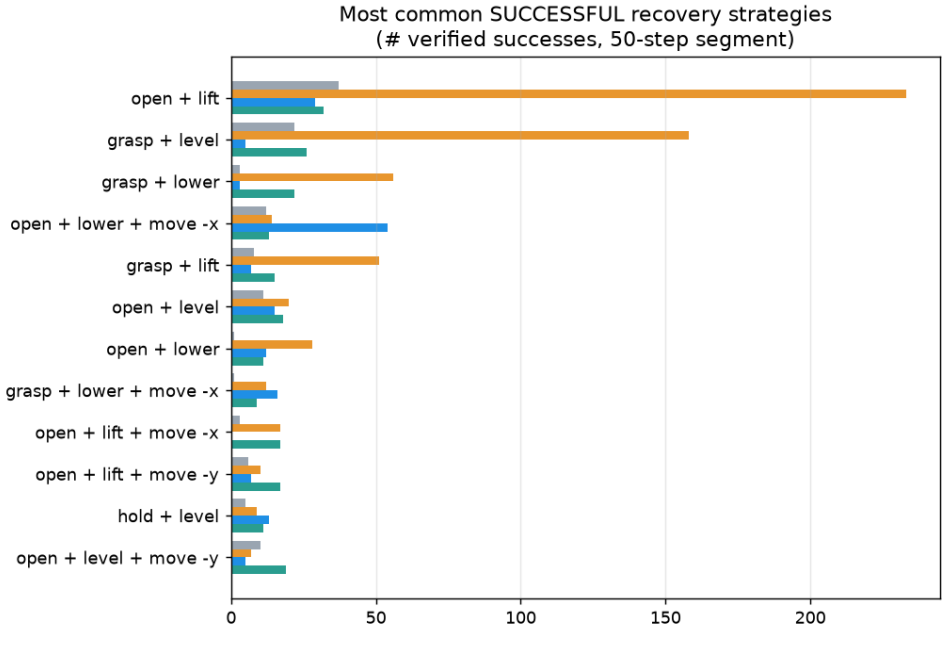
NEW: MAP-Elites over 60 strategy cells (gripper × motion)

- WM diverse: 2.8× more verified recoveries per simulator rollout than random (0.171 vs 0.062), p = 4e-9. Drawer: 0.43 vs 0.07 solved
- noise-CEM is the smoothest and closest to SmolVLA, but WORSE than random: it inherits SmolVLA's 'keep grasping' habit (drawer 0.02)
- quality-diversity: most distinct strategies per solved failure (4.0 vs 2.2), still beats random (p = 0.002)

Which strategies the WM finds



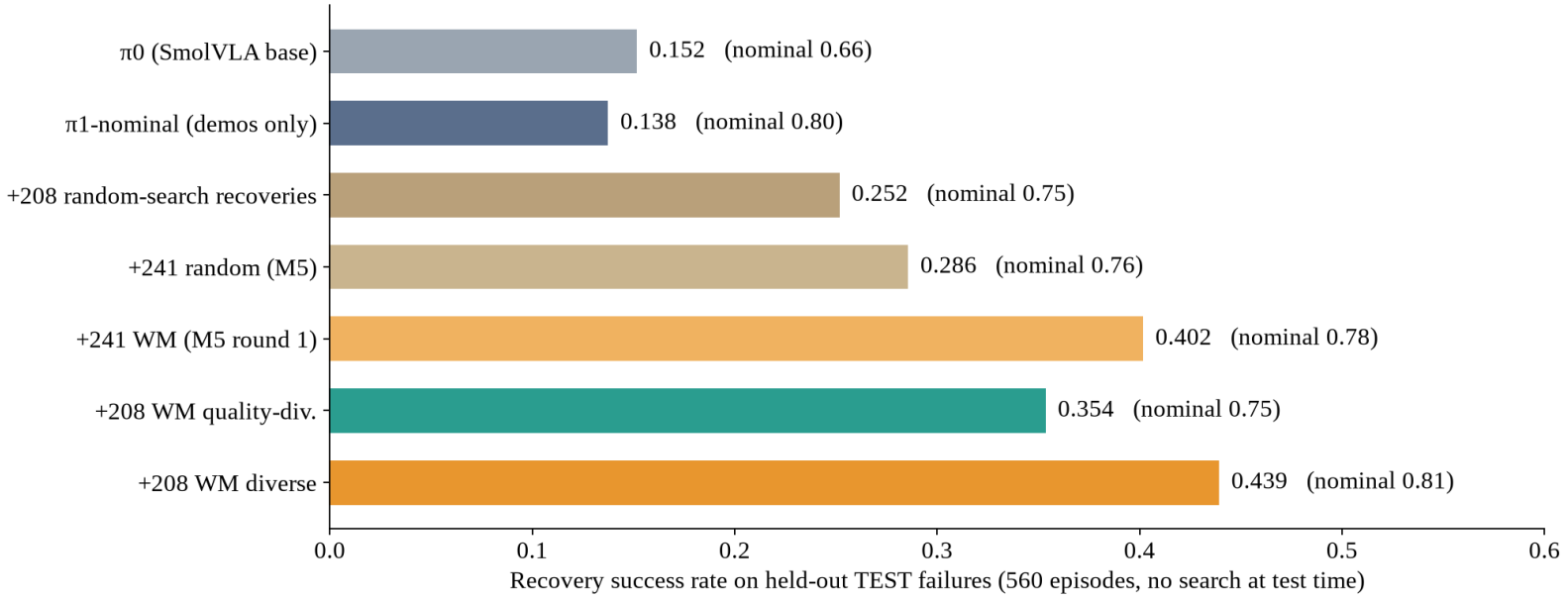




Top recovery = open + lift: let go, get clear, let SmolVLA re-approach as from a normal start. WM found it ~230×, random ~35×.

Drawer: random spends 55% of sims on 'grasp again' (0 / 732 succeed); WM spends 83% on 'stay open, lift' (79 successes).

M5c (Spatial, seed 0): WM data > random data for the VLA



0.439

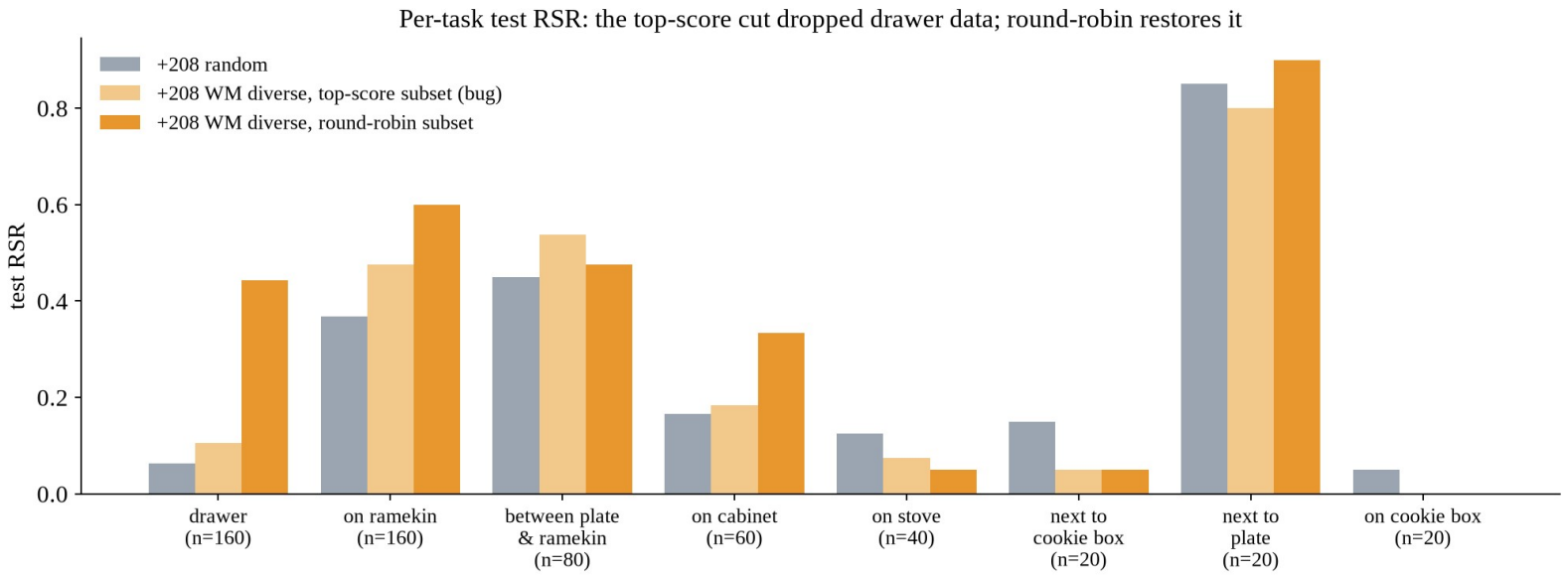
test RSR, +208 WM recoveries (vs 0.252 random, same count, same init)

0.808

nominal SR: no longer lost (demo-only SFT: 0.796)

- M5 (241 vs 241): WM 0.402 vs random 0.286, +0.116 [+0.045, +0.191] scene-bootstrap CI, paired McNemar p = 4e-7
- M5c (208 vs 208, round-robin subset): WM diverse 0.439, quality-div. 0.354, random 0.252
- more is not better: all 423 WM trajectories 0.389 < 208 balanced. One training seed per variant

A data-selection bug that hid the gain



What went wrong

- to match counts we kept the top-208 trajectories by critic score
- the critic scores drawer recoveries low, so only 4 drawer trajectories survived (of 44)
- drawer test RSR collapsed to 0.11
- fix: round-robin per failure (each failure's best, then 2nd best, ...): drawer 0.44, overall 0.30 → 0.44

Lesson: the distribution of the distilled data matters as much as the search